AI 388 / CS 388 Natural Language Processing

Modules 1–7: Complete Study Guide

All 22 exam topics in lecture (module) order

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide 1

Module 1: Linear Classification 7

1 Bag-of-Words Features 8
- 1.1 Setup and notation (lecture) . . . 8
- 1.2 The feature-extraction pipeline . . . 8
- 1.3 Geometry: what the feature space looks like . . . 9
- 1.4 Which preprocessing shrinks the feature space? . . . 10
- 1.5 Exercise 1.0 (Canvas) . . . 11
- 1.6 Practice . . . 11

4 Sentiment Analysis: Useful Features 12
- 4.1 What makes sentiment hard (lecture examples) . . . 12
- 4.2 Useful features (what the exam can ask) . . . 12
- 4.3 History (lecture 1.4.0) — MC fodder . . . 13
- 4.4 A1 connection . . . 13
- 4.5 Past exam problems . . . 13
- 4.6 Practice . . . 14

2 Binary Perceptron: Algorithm and Loss 15
- 2.1 Algorithm (lecture form) . . . 15
- 2.2 Worked trace: the lecture example . . . 15
- 2.3 Non-separable data and the fix . . . 16
- 2.4 Perceptron as SGD on a loss . . . 17
- 2.5 Past exam problems . . . 17
- 2.6 Practice . . . 18

3 Binary Logistic Regression 20
- 3.1 Model . . . 20
- 3.2 Training objective . . . 20
- 3.3 Gradient derivation (be able to reproduce) . . . 21
- 3.4 Worked numeric example . . . 21
- 3.5 Past exam / exercise problems . . . 22
- 3.6 Practice . . . 23

7 Optimization: SGD, Step Size, Initialization **24**
7.1 The optimization problem . . . 24
7.2 Step size: the quadratic example . . . 24
7.3 Choosing step sizes (lecture recipe) . . . 25
7.4 Regularization (lecture: "don't really use") . . . 26
7.5 Initialization (neural networks) . . . 26
7.6 Practice . . . 27

Module 2: Multiclass and Neural Classification **28**

5 Multiclass Classification: Weights and Features **29**
5.1 Setup . . . 29
5.2 Example multiclass tasks (lecture 2.0.2) . . . 30
5.3 Practice . . . 30

6 Multiclass Perceptron (and Multiclass Logistic Regression) **32**
6.1 Multiclass perceptron . . . 32
6.2 Multiclass logistic regression (softmax regression) . . . 33
6.3 Practice . . . 34

8 Feedforward Neural Networks: Definition and Initialization **35**
8.1 Definition . . . 35
8.2 Why it works: the XOR example (lecture 2.1.0) . . . 36
8.3 Parameter counting and shape checking . . . 36
8.4 Initialization . . . 37
8.5 Practice . . . 37

9 Training Neural Networks **39**
9.1 Objective . . . 39
9.2 Backpropagation . . . 39
9.3 Implementation in PyTorch (lecture 2.2.1, A2) . . . 40
9.4 Practical training techniques (lecture 2.2.2) . . . 41
9.5 Practice . . . 41

Module 3: Word Embeddings **42**

10 Word Embeddings: Skip-gram, Properties, Bias **43**
10.1 Skip-gram model . . . 43

10.2 Other embedding methods (3.1.1) . . . 45
10.3 Properties of embeddings . . . 45
10.4 Bias in word embeddings (3.1.2) . . . 46
10.5 Long skip-gram problems . . . 47
10.6 Practice . . . 48

11 Deep Averaging Networks (Assignment 2) 49
11.1 Model . . . 49
11.2 Assignment 2 implementation details . . . 49
11.3 Limitations (exam favorite) . . . 50
11.4 Past exam problems . . . 51
11.5 Practice . . . 51

Module 4: Language Modeling and Self-Attention 52

12 n-gram Language Modeling (+ Smoothing, Perplexity, Neural LMs) 53
12.1 Definition . . . 53
12.2 Uses (lecture) . . . 54
12.3 Smoothing: absolute discounting . . . 54
12.4 Evaluation: perplexity . . . 55
12.5 Neural LMs and RNNs (background for Topic 13, Topic 15) . . . 55
12.6 Practice . . . 56

13 Self-Attention: Definition and Mechanics 57
13.1 Attention with one query (lecture 4.2.0) . . . 57
13.2 Self-attention (lecture 4.2.1) . . . 57
13.3 Multi-head self-attention (lecture 4.2.2) . . . 59
13.4 Positional encodings (lecture 4.2.3) . . . 59
13.5 Practice . . . 60

Module 5: Transformers and Decoding 61

14 Transformers: Architecture Details 62
14.1 One Transformer layer (encoder block) . . . 62
14.2 Parameter counting . . . 63
14.3 Using Transformers (lecture 5.0.1) . . . 63
14.4 A3 Part 1: letter counting . . . 63
14.5 Efficiency extensions (lecture 5.0.3, MC fodder) . . . 64

14.6 Practice . . . 64

15 Transformer Language Modeling (Assignment 3) 65
15.1 Model . . . 65
15.2 Training . . . 65
15.3 The causal mask . . . 66
15.4 Inference and scoring (A3 `NeuralLanguageModel`) . . . 66
15.5 Evaluation: perplexity on A3 . . . 66
15.6 Your A3 model, piece by piece . . . 66
15.7 Past exam problems . . . 67
15.8 Practice . . . 68

18 Decoding: Greedy, Beam Search, Nucleus Sampling 69
18.1 Exhaustive vs. greedy vs. beam . . . 69
18.2 Sampling and nucleus sampling . . . 70
18.3 Past exam: Fall 2023 Q1(16–18) . . . 70
18.4 Practice . . . 71

Module 6: Pre-training and Tokenization 72

16 Encoders: Masked Language Modeling and BERT 73
16.1 Why not a Transformer LM? Why masking? . . . 73
16.2 BERT pre-training . . . 73
16.3 Applying BERT: fine-tuning . . . 74
16.4 Encoder vs. decoder vs. encoder–decoder . . . 74
16.5 Past exam / exercises . . . 75
16.6 Practice . . . 76

17 Subword Tokenization (BPE, WordPiece) 77
17.1 Byte Pair Encoding (BPE; Sennrich et al. 2016) . . . 77
17.2 WordPiece / unigram LM (SentencePiece) . . . 77
17.3 Greedy (longest-match) tokenization, the exam algorithm . . . 78
17.4 Practice . . . 79

Module 7: POS Tagging, HMMs, Parsing 80

19 POS Tagging and Its Ambiguities 81
19.1 Tags (lecture crash course) . . . 81

19.2 The "Fed raises interest rates 0.5 percent" example . . . 81
19.3 Tagging with classifiers (lecture 7.1.0) . . . 82
19.4 Exercises . . . 82
19.5 Practice . . . 83

20 Hidden Markov Models: Definition, Estimation, Viterbi **84**
20.1 Definition . . . 84
20.2 Parameter estimation: count and normalize . . . 85
20.3 Inference: Viterbi . . . 85
20.4 HMMs for POS tagging in practice . . . 86
20.5 Practice . . . 87

21 Constituency Syntax: Trees and Ambiguity **88**
21.1 What trees look like . . . 88
21.2 Ambiguities (must recognize) . . . 89
21.3 Constituency tests (how linguists justify phrases) . . . 89
21.4 Practice . . . 90

22 PCFGs, Binarization, Markovization, CKY **91**
22.1 CFG and PCFG . . . 91
22.2 From treebank to grammar . . . 92
22.3 Binarization . . . 92
22.4 Markovization (refining grammars) . . . 92
22.5 CKY . . . 93
22.6 Practice . . . 94

MODULE 1

Linear Classification

TOPIC 1

Bag-of-Words Features

Sources: Lectures 1.1.0 Linear Binary Classification, 1.1.1 Sentiment Analysis & Basic Feature Extraction, 1.4.0 Sentiment Analysis; slides seg-1, seg-2, seg-7; Exercise 1.0; Assignment 1; Fall 2022 Q1(4); Fall 2023 Q2(a), Q1(15).

In plain words

A classifier only understands numbers, but our input is a string. A *feature extractor* f turns the string into a long vector. In **bag-of-words** (BoW), the vector has one slot per vocabulary word. A slot holds how many times that word appears (or just 1/0 for present/absent). Word order is thrown away: the sentence becomes a "bag" of words. Each vocabulary word is one *axis* of the feature space, so a sentence is a point in $\mathbb{R}^{|V|}$.

1.1 Setup and notation (lecture)

- Input $\mathbf{x}$ (usually a string). Feature extractor f: string $\to f(\mathbf{x}) \in \mathbb{R}^d$.
- Label $y \in \{-1, +1\}$ for binary classification (negative/positive).
- Classifier = weight vector $\mathbf{w} \in \mathbb{R}^d$ (some texts write θ). Same dimension d as the features.
- Decision: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$. The decision boundary $\{f(\mathbf{x}) : \mathbf{w}^\top f(\mathbf{x}) = 0\}$ is perpendicular to $\mathbf{w}$.
- **No separate bias.** Instead of $\mathbf{w}^\top f(\mathbf{x}) + b$, append a constant feature 1: $f(\mathbf{x}) = [3, -1, 2] \mapsto [3, -1, 2, 1]$. The last weight then plays the role of b. (The lecture: "we are going to not use [b].")

1.2 The feature-extraction pipeline

Raw string → feature vector (lecture 1.1.1, five steps)

1. **Tokenization.** Split the string into tokens. Whitespace splitting leaves `great!` and `great` as different words. A tokenizer separates punctuation and contractions: `great!`→`great !`, `wasn't`→`was n't`.
2. **[Sometimes] Stop-word removal.** Drop function words (*the, of, a*). These skew the vectors and carry little sentiment.
3. **[Sometimes] Casing.** Lowercase everything, or *truecase* (e.g. for speech-recognizer output that has no case).
4. **Unknown words.** Words outside the vocabulary (e.g. the 10,000 most frequent) become `UNK`, or are simply dropped for sentiment.
5. **Indexing.** Map each word or n-gram to an integer position in the vector, using a map (A1's `Indexer`): `the`→5, `great`→247, ...

1.2.1 Count vs. presence features

- **Counts:** the value is how many times the word occurs ("the" twice ⇒ 2).
- **Presence/absence (indicator):** the value is 1 if the word occurs at all, else 0.

For "the movie was great" with a 10,000-word vocabulary there are **four 1s and 9,996 zeros**, so the vector is extremely *sparse*. In code you store only the nonzero entries (A1 suggests a `Counter` mapping index→value). The weight vector itself is a dense numpy array.

1.2.2 Bag-of-n-grams

An n-gram is a sequence of n consecutive words. The 2-grams (bigrams) of "the movie was great" are *the movie, movie was, was great*. A sentence with L tokens has $L - n + 1$ n-grams (more if you add start/end tokens). Bigrams can capture phrases like *not great* and *watch again*, which unigrams cannot. The cost is a much larger feature space (e.g. about 100,000 bigrams instead of 10,000 words). A1 tip: encode mixed feature types with "magic strings" in one indexer, e.g. `Unigram=great` and `Bigram=great|movie`.

1.2.3 TF-IDF (introduced, not used much)

TF-IDF

$$\begin{aligned} \text{tf}(w,d) &= \text{count of } w \text{ in document } d \\ \text{idf}(w) &= \log \frac{\text{total \# documents}}{\text{\# documents containing } w} \\ \text{tf-idf}(w,d) &= \text{tf}(w,d) \times \text{idf}(w) \end{aligned}$$

A common word (*the*) appears in almost every document, so idf $\approx \log 1 = 0$ and its value shrinks toward 0. A rare word like *mango* appearing in 1 of 10,000 documents gets $\log_{10}(10000/1) = 4$ (the lecture's example uses base 10). The effect: emphasize words characteristic of this document.

1.3 Geometry: what the feature space looks like

Each feature is an axis. With vocabulary [good, bad, not], "not bad" $\mapsto (0,1,1)$ is a corner of the unit cube. A linear classifier is a plane through the origin (no bias feature) separating the $+$ points from the $-$ points. Changing the feature set changes the geometry. Adding bigram axes (*not good, not bad*) can make inseparable data separable (Topic 2, the good/bad/not good/not bad example).

Worked example: building vectors

Vocabulary, in this order: [a, bad, good, great, movie, not, the, was]. Lowercase and tokenize; drop words not in the vocabulary.
Sentence: *The movie was not bad, the acting was great!* Tokens: the movie was not bad , the acting was great ! In-vocabulary tokens: the(2) movie(1) was(2) not(1) bad(1) great(1). "acting", "," and "!" are dropped.

$$\text{count } f(\mathbf{x}) = [\,0,1,0,1,1,1,2,2\,], \qquad \text{presence } f(\mathbf{x}) = [\,0,1,0,1,1,1,1,1\,].$$

Bigrams of *the movie was not bad* (5 tokens $\Rightarrow$ 4 bigrams): the movie, movie was, was not, **not bad**. With a bias feature appended, the count vector becomes [0,1,0,1,1,1,2,2,1].

Worked example: TF-IDF

Four documents: d_1 "the movie was great", d_2 "the movie was bad", d_3 "the plot was great great", d_4 "a great cast".

word	docs containing	idf $= \log_{10}(4/\text{df})$	idf with ln
the	3	$\log_{10}(4/3) = 0.1249$	0.2877
great	3	0.1249	0.2877
bad	1	$\log_{10} 4 = 0.6021$	1.3863

tf-idf of *great* in $d_3 = 2 \times 0.1249 = 0.2499$ (base 10). *bad* in d_2 gets 1×0.6021, more than twice as large, even though it occurs once. If a question doesn't state the log base, say which one you used.

1.4 Which preprocessing shrinks the feature space?

Midterm Fall 2022 Q1(4)

Unigram BoW model. Which preprocessing techniques reduce the size of the feature space? I. lowercasing; II. bigrams instead of unigrams; III. stemming; IV. removing stopwords; V. tokenizing punctuation (`great!`→`great !`); VI. tf-idf weighting.

Official answer: I, III, IV, V.

Why: I merges *Great*/*great*. III merges *loved*/*love*. IV deletes features. V removes `great!`, `great.`, `great,...` as separate features, adding only a few punctuation tokens. II *increases* the space. VI changes the *values* in the vector, not the number of dimensions.

Midterm Fall 2023 Q2(a)

Vocabulary [good, great, not]. Examples: `goodx` +; `great` +; `not greatq` −; `not good` − (typos are deliberate). Write the feature vectors.

Official answer: [0,0,0], [0,1,0], [0,0,1], [1,0,1].

Why: `goodx`≠`good` and `greatq`≠`great`: an exact string match fails, so a typo makes the word vanish. The first example becomes the all-zero vector. (This motivates subword tokenization, Topic 17, and A2's typo setting.)

Midterm Fall 2023 Q1(15) (also see Topic 17)

Train a sentence classifier on data with only singular nouns, then test on data with plural nouns. Which could help generalization? I. stemming; II. subword tokenization; III. pre-trained word embeddings; IV. BoW featurization; V. running a syntactic parser.

Official answer: I, II, III.

Why: Stemming maps *dogs*→*dog*. Subwords give *dog* + *s*, sharing the *dog* piece. Pre-trained embeddings put *dogs* near *dog*. BoW alone treats *dogs* as an unseen feature. A parser doesn't obviously help.

1.5 Exercise 1.0 (Canvas)

Canvas Exercise 1.0 Q1–Q2

(1) "the movie was good" + / "the movie was bad" −. Which two features are most helpful?

Answer: Indicator(contains "good"), Indicator(contains "bad"). Either one alone is enough.

(2) "the movie was good" + / "not good" − / "not not good" +. Which further feature helps?

Answer: Indicator(Count("not") is odd). The label flips with each *not*. A plain Count("not") is monotone in the number of *not*s, so a linear model can't use it to make the label alternate.

Traps

- **Follow the stated vocabulary order** when writing vectors. Auto-graded answers compare the exact list.
- **Out-of-vocabulary words contribute nothing** (all-zero vector possible). A typo makes the word out of vocabulary.
- Count vs. indicator: read the question. "Bag-of-words" in lecture defaults to counts; A1 code (and many exam questions) use 0/1.
- Bias feature: include it only if the question says so. If you include it, it's usually the last coordinate, always 1.
- BoW is *order-blind*: "good, not bad" and "bad, not good" get identical unigram vectors.
- Number of n-grams in L tokens is $L - n + 1$ (no padding tokens).

1.6 Practice

Practice

1. Vocabulary [bad, boring, fun, not, very]. Give count vectors for (a) "very very fun" (b) "not boring , not bad" (c) "fantastic".
2. How many bigrams does a 12-token sentence have? How many trigrams?
3. 1,000 documents. "excellent" appears in 10 of them; in document d it occurs 3 times. Compute tf-idf with $\log_{10}$.
4. True/False: applying tf-idf weighting reduces the dimensionality of the BoW feature space.
5. Give two sentences with identical unigram BoW vectors but opposite sentiment. Which feature type fixes this?

Answers

1. (a) [0,0,1,0,2] (b) [1,1,0,2,0] (c) [0,0,0,0,0]. 2. 11 bigrams, 10 trigrams. 3. $3 \times \log_{10}(1000/10) = 3 \times 2 = 6$. 4. False: same dimensions, different values. 5. e.g. "it was good , not bad" vs. "it was bad , not good". Bigram features (*not bad* vs. *not good*) separate them.

TOPIC 4

Sentiment Analysis: Useful Features

Sources: Lectures 1.1.1 Sentiment Analysis & Basic Feature Extraction, 1.4.0 Sentiment Analysis; slides seg-2, seg-7; Exercise 1.0; A1 Part 3; Fall 2022 Q2(c); Fall 2023 Q2(e), Q1(5–6).

In plain words

Sentiment is mostly carried by *content words* (great, awful, enjoyable) and short phrases (*watch again*, *not good*). Unigram bag-of-words captures the words, and bigrams capture short phrases and local negation. What BoW misses is structure: negation that scopes over several words, and discourse markers like "but" that override what came before. Simple BoW classifiers were surprisingly strong for years. Large pre-trained models (BERT etc.) now dominate.

4.1 What makes sentiment hard (lecture examples)

example	phenomenon
the movie was great! would watch again! (+)	polar word *great*; phrase *watch again*
the film was awful; I'll never watch again (−)	*watch again* appears in a negative example: negation (*never*)
the movie was gross and overwrought, but I liked it (+)	discourse: *but* overrides earlier negative words
this movie was not really very enjoyable (−)	negation scopes over several words; unigram *enjoyable* misleads

4.2 Useful features (what the exam can ask)

1. **Unigram indicators/counts** of sentiment words. Strong baseline.
2. **Bigrams** (and *n*-grams): *not good*, *watch again*, *not bad*. They capture local negation and phrases.
3. **Negation marking:** after a *not*, extract a "not_X" feature for every following word X: *not really*, *not very*, *not enjoyable*. Without a syntactic parser you can't determine the negation scope reliably.
4. **Presence vs. frequency:** the 2002 study (Pang, Lee & Vaithyanathan) compared both. Presence (0/1) is a common choice for sentiment.
5. **Preprocessing:** lowercasing and stop-word removal often help sentiment. *Keep negation words*: the A1 `BetterFeatureExtractor` removes spaCy stop words *except* {no, nor, not, never}.
6. Others suggested in A1: tf-idf weighting, clipping word frequencies, discarding rare words, first-word features, other *n*-grams.
7. **Task-specific indicator features**, e.g. "count of *not* is odd" (Exercise 1.0).

4.3 History (lecture 1.4.0) — MC fodder

- **Pang, Lee & Vaithyanathan (2002)**: simple feature sets do pretty well. Best were unigram and unigram+bigram features with Naive Bayes, MaxEnt (=LR) and SVMs.
- **Wang & Manning (2012)**: revisited BoW. A modified Naive Bayes (NB-SVM) matched or beat a 2012 neural recursive autoencoder.
- **Kim (2014)**: CNNs beat these substantially.
- Today: large pre-trained networks (BERT, XLNet, . . .) on SST binary classification, going from about 90 (BiLSTMs) to about 97.

4.4 A1 connection

Binary SST sentences (neutral removed); labels 0/1. Required dev accuracy: unigram perceptron $\geq 74\%$, unigram LR $\geq 77\%$. Q3: bigram indicators on adjacent word pairs. Q4: a "better" extractor that is more than just unigrams+bigrams. Always-positive baseline: 444/872 = 50.9% dev.

4.5 Past exam problems

Midterm Fall 2022 Q2(c)

Your unigram BoW model has been in production for years. A new, very negative slang word appears that the model has never seen. Name two ways to update the model.

Official answer: (1) Add labeled sentences containing the word and retrain (or continue training) for a few epochs. (2) Directly add a feature for the word with a large negative weight.

Midterm Fall 2023 Q2(e)

Typos can be anywhere in a word. Which is more effective for robustness: subword tokenization or explicit typo repair (closest vocabulary word by edit distance)?

Official answer: Typo repair is generally less fragile. Subword tokenization breaks typo'd words into odd pieces (`greqat`→ gre q at), while edit-distance repair maps `goodx`→`good`.

Midterm Fall 2023 Q1(5)–(6) (preview of Topic 11, Topic 12)

A unidirectional RNN reads "The play was terrible . . . lackluster . . . unconvincing . . . At least the dinner was excellent." and predicts very positive. Why?

Official answer: D: an RNN's state is heavily influenced by recent tokens. Q6 (what might work better, select all):

Official answer: I (BoW: more negative words overall), II (DAN), III (Transformer). All are order-insensitive or attend globally.

Traps

- Removing stop words can delete *not/no/never* unless you exclude them. That destroys negation features.
- Bigrams increase the feature space. They help negation only locally (*not really very enjoyable* has no *not enjoyable* bigram).

- Both “Indicator(contains not)” and “Count(not)” are only partially useful for alternating negation. The right feature depends on the data pattern (Exercise 1.0).

4.6 Practice

Practice

1. List the features extracted by the “not_X” trick for “it was not very funny at all”.
2. Design one feature that lets a linear model handle “X, but Y” sentences where Y decides the label.
3. Why might presence features beat counts for short sentiment snippets?

Answers

1. not_very, not_funny, not_at, not_all (every word after *not*; you might stop at punctuation).
2. e.g. prefix every word after *but* with `AFTERBUT_` (or weight post-*but* words more), so the model learns separate weights for clause-final sentiment.
3. Repeating a word rarely makes the sentiment stronger in proportion. Counts let frequent but neutral repetitions dominate the score.

TOPIC 2

Binary Perceptron: Algorithm and Loss

Sources: Lectures 1.2.0 Perceptron, 1.2.1 Perceptron as Minimizing Loss, 1.1.2 Basics of Learning, Gradient Descent; slides seg-3, seg-4, seg-5; Exercise 1.1; A1 Part 1; Fall 2022 Q3; Fall 2023 Q1(1), Q2(b).

In plain words

Go through the examples one at a time. If the current weights classify an example correctly, do nothing. If they get a positive example wrong, **add** its feature vector to $\mathbf{w}$, which pushes its score up next time. If they get a negative example wrong, **subtract** its feature vector. Repeat for several passes (epochs). If some linear boundary separates the data, this provably stops making mistakes. If not, it cycles forever.

2.1 Algorithm (lecture form)

Perceptron

Decision rule (lecture): $y_{\text{pred}} = +1$ if $\mathbf{w}^\top f(\mathbf{x}) > 0$, else -1. A score of exactly 0 is **negative**.
Initialize $\mathbf{w} = \mathbf{0}$.
for $t = 1 \dots$ num_epochs:
  for $i = 1 \dots D$:
    $y_{\text{pred}} \leftarrow \text{decision}(\mathbf{w}^\top f(\mathbf{x}^{(i)}))$
    if $y_{\text{pred}} = y^{(i)}$: no change
    else if $y^{(i)} = +1$: $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, f(\mathbf{x}^{(i)})$
    else $(y^{(i)} = -1)$: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\, f(\mathbf{x}^{(i)})$
Compact form of the update on a mistake: $\boxed{\mathbf{w} \leftarrow \mathbf{w} + \alpha\, y^{(i)}\, f(\mathbf{x}^{(i)})}$.

- $\mathbf{w}$ and $f(\mathbf{x})$ live in the same d-dimensional space, so they can be added.
- **Step size α doesn't matter if $\mathbf{w}$ starts at 0 and α is constant.** Every update is a multiple of α, so $\mathbf{w} = \alpha \cdot (\text{integer combination of features})$. The sign of every score, and hence every decision, is unchanged. Exams often say "assume $\alpha = 1$". (This is a consequence of the algorithm, not a lecture statement. It fails if $\mathbf{w}$ starts nonzero.)
- In practice: shuffle the data each epoch (A1 handout) and run a fixed number of epochs.
- **A1 code form** uses labels $y \in \{0, 1\}$ and predictions $\hat{y} \in \{0, 1\}$: `w += lr * (label - prediction) * value`. Here $(\text{label} - \hat{y}) \in \{-1, 0, +1\}$, so this is exactly the same algorithm.

2.2 Worked trace: the lecture example

Worked example: lecture 1.2.0 (features: Movie, Good, Bad, Not; $\alpha = 1$; rule > 0)

Data: "movie good" $+1$ [1,1,0,0]; "movie bad" -1 [1,0,1,0]; "not good" -1 [0,1,0,1].

epoch	example	y	score $\mathbf{w}^\top f(\mathbf{x})$	pred	$\mathbf{w}$ after
1	movie good	$+1$	0	-1 wrong	[1,1,0,0]
1	movie bad	-1	1	$+1$ wrong	[0,1,-1,0]
1	not good	-1	1	$+1$ wrong	[0,0,-1,-1]
2	movie good	$+1$	0	-1 wrong	[1,1,-1,-1]
2	movie bad	-1	0	-1 ok	[1,1,-1,-1]
2	not good	-1	0	-1 ok	[1,1,-1,-1]
3	all three		2, 0, 0	all ok	**converged**

Learned: *bad* and *not* negative, *movie* and *good* positive. Two examples sit exactly at score 0 and are classified negative only because of the tie rule.

How to trace a perceptron quickly (exam procedure)

1. Write every $f(\mathbf{x})$ as a row in the given feature order. Note the tie rule (>0 or ≥ 0) and α.
2. Keep a table: score $\rightarrow$ prediction $\rightarrow$ correct? $\rightarrow$ new $\mathbf{w}$.
3. On a mistake add $\alpha f(\mathbf{x})$ ($y=+1$) or subtract it ($y=-1$). Otherwise copy $\mathbf{w}$ unchanged.
4. "One epoch" means one pass in the given order. "Until convergence" means repeat until a full pass has no mistakes.
5. Sanity check: the final $\mathbf{w}$ is always $\alpha \sum(\pm\text{features of mistaken examples})$.

2.3 Non-separable data and the fix

Worked example: lecture's XOR-like example

Features [good, bad, not]: "good" + [1,0,0]; "bad" − [0,1,0]; "not good" − [1,0,1]; "not bad" + [0,1,1].
Suppose some $\mathbf{w}$ classified all four correctly (with the >0 rule): $w_g > 0$, $w_b \leq 0$, $w_g + w_n \leq 0$, $w_b + w_n > 0$. The last two give $w_n \leq -w_g < 0$ and $w_n > -w_b \geq 0$. That's a contradiction, so the data are **not linearly separable** and the perceptron **loops forever**.
One epoch from $\mathbf{w}=0$ (Exercise 1.1 Q2): [1,0,0]→[1,0,0]→[0,0,-1]→[0,1,0]. The final answer is **[0,1,0]**, and further epochs keep cycling.
Fix with features: add bigram features *not good*, *not bad*. In 5-D the data become separable, e.g. $\mathbf{w}$ with $w_{\text{not bad}}$ large positive and $w_{\text{not good}}$ large negative. **Fix with model:** a nonlinear classifier (feed-forward NN, Topic 8). Switching to logistic regression or adding epochs does *not* help (Exercise 1.1 Q3).

Convergence facts

- If the data are linearly separable, the perceptron converges (finds a separating $\mathbf{w}$) in finitely many updates.
- If not separable, it never converges. Weights cycle and accuracy fluctuates.
- It is **not** always guaranteed to converge (Fall 2023 Q1(1) statement I is false).
- With many features (e.g. big BoW vocabularies) data are often separable in practice. Feature design is what changes separability.

2.4 Perceptron as SGD on a loss

Perceptron loss (lecture 1.2.1)

$$\mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}) = \begin{cases} 0 & \text{if the prediction is correct} \\ -y^{(i)}\,\mathbf{w}^\top f(\mathbf{x}^{(i)}) & \text{otherwise} \end{cases} \qquad \text{i.e.} \quad \max\left(0,\, -y^{(i)}\,\mathbf{w}^\top f(\mathbf{x}^{(i)})\right)$$

Gradient for $y = +1$: $\nabla_{\mathbf{w}}\mathcal{L} = 0$ if correct, $-f(\mathbf{x})$ if wrong. SGD step $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla\mathcal{L} = \mathbf{w} + \alpha f(\mathbf{x})$, exactly the perceptron update. Symmetric for $y = -1$: gradient $+f(\mathbf{x})$, update $\mathbf{w} - \alpha f(\mathbf{x})$.

Plotted against $z = \mathbf{w}^\top f(\mathbf{x})$ for a positive example, the loss is flat at 0 for $z \geq 0$ and a line of slope -1 for $z < 0$. It is a "hinge" at 0, convex, and piecewise linear. Lecture visualization: in weight space the total loss is 0 inside a wedge of separating weight vectors. SGD walks downhill to that flat plateau, which is convergence. Removing a point that doesn't constrain the boundary leaves the wedge unchanged.

General SGD template (lecture 1.1.2)

Objective: $\sum_{i=1}^{D} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w})$, linear over examples. Loop: for each epoch, for i up to D: sample $j \in [1, D]$, $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{x}^{(j)}, y^{(j)}, \mathbf{w})$. The gradient points toward higher loss, so subtract it. Perceptron, logistic regression and neural networks are all instances (Topic 7).

2.5 Past exam problems

Midterm Fall 2022 Q3(a) — rule $\mathbf{w}^\top\mathbf{x} > 0$

Points (x_1, x_2, y): $(1, 0, +)$, $(1, 1, +)$, $(0, 1, -)$. One pass from $\mathbf{w} = 0$. Final $\mathbf{w}$?

Solution. $(1,0)$: score $0 \Rightarrow$ pred $-$, wrong $\Rightarrow \mathbf{w} = (1,0)$. $(1,1)$: score $1 \Rightarrow +$, ok. $(0,1)$: score $0 \Rightarrow -$, ok.

Official answer: $\boxed{[1, 0]}$.

Midterm Fall 2022 Q3(b)

Add a 4th, *negative* point. Describe the region where the perceptron will *not* converge.

Solution. No bias, so the boundary passes through the origin. Feasible $\mathbf{w}$ for the 3 points (rule > 0): $w_1 > 0$ (from $(1,0)+$), $w_1 + w_2 > 0$ (from $(1,1)+$), $w_2 \leq 0$ (from $(0,1)-$). So $\mathbf{w}$ ranges from direction $(1,0)$ to just short of $(1,-1)$. A new negative $\mathbf{x}$ makes the problem infeasible iff $\mathbf{w}^\top\mathbf{x} > 0$ for *every* feasible $\mathbf{w}$. At $\mathbf{w} = (1,0)$ that needs $x_1 > 0$; as $\mathbf{w} \to (1,-1)$ it needs $x_1 - x_2 \geq 0$.

Official answer: $x_2 \leq x_1$ and $x_1 > 0$. Picture: the boundary can rotate like a clock hand from "noon" (vertical) to "1:30" (45°) with negatives on the left. Any negative point to the right of every such boundary breaks separability.

Midterm Fall 2022 Q3(c)

Add $(4, 2, -)$. Does any neural network classify all points correctly?

Official answer: Yes. Neural networks are universal function approximators, and these 4 points (no

two identical with different labels) can be separated by a nonlinear function.

Midterm Fall 2023 Q2(b) — this exam uses $\mathbf{w}^\top f(\mathbf{x}) \geq 0$ (0 is positive)

Vectors from Q2(a): [0,0,0]+, [0,1,0]+, [0,0,1]− (greatq is unknown, so only *not* fires), [1,0,1]−. With rule ≥ 0: ex1 score 0 $\to$ + ok; ex2 0 $\to$ + ok; ex3 0 $\to$ + **wrong** $\Rightarrow$ $\mathbf{w} = [0, 0, -1]$; ex4 score $-1 \to -$ ok.

Official answer: $\boxed{[0, 0, -1]}$. **Contrast (lecture rule > 0):** ex1 wrong but $f(\mathbf{x}) = 0$ so no change; ex2 wrong $\Rightarrow$[0,1,0]; ex3 score 0 $\to -$ ok; ex4 $-$ ok $\Rightarrow$ [0,1,0]. The tie rule changes the answer, so **always use the rule the question states**.

Midterm Fall 2023 Q1(1) (4 pts, select all)

I. Perceptron always converges. II. LR with BoW features is convex. III. SGD can optimize LR. IV. Perceptron = SGD with a certain loss. V. As epochs$\to \infty$, LR and perceptron give the same boundary. VI. Same, if data are separable.

Official answer: II, III, IV. VI is false: both may classify the training points identically, but the actual weight vectors and boundaries differ.

Canvas Exercise 1.1 Q1

Features [good, bad, not]; "good" +, "not good" −, "bad" −; rule > 0; run to convergence.

Solution. Epoch 1: [1,0,0], then score 1 on [1,0,1] $\Rightarrow$ wrong $\Rightarrow$[0,0,-1]; "bad" score 0 ok. Epoch 2: "good" score 0 wrong $\Rightarrow$[1,0,-1]; others ok. Epoch 3: no mistakes.

Answer: [1,0,-1].

Traps

- **Tie rule:** lecture uses > 0 ($0 \to$ negative). Fall 2022 uses > 0; Fall 2023 uses ≥ 0. Use whatever the question says.
- A mistaken example with $f(\mathbf{x}) = \mathbf{0}$ produces no change (adding the zero vector).
- Update only on mistakes. Logistic regression updates on every example (Topic 3).
- Don't confuse "one epoch" with "until convergence".
- No bias feature means the boundary passes through the origin. That matters for separability questions like Fall 2022 Q3(b).

2.6 Practice

Practice

1. Features [great, awful, plot]. Data: "great plot" +, "awful plot" −, "plot" −, "great" +. Rule > 0, $\alpha = 1$, $\mathbf{w} = 0$. Trace to convergence.
2. Same data, $\alpha = 0.5$. What changes?
3. Same data, but start at $\mathbf{w} = [0, 0, 1]$, $\alpha = 1$. Trace one epoch.
4. Write the perceptron loss for a negative example and its gradient in both cases.
5. Are these 2-D points separable by a line through the origin: $(1, 1)+$, $(-1, -1)+$, $(1, -1)-$?

Explain.

Answers

1. Ep1: [1,0,1] score 0 → wrong ⇒[1,0,1]; [0,1,1] score 1 wrong ⇒[1,-1,0]; [0,0,1] score 0 ok; [1,0,0] score 1 ok. Ep2: scores 1, −1, 0, 1, all correct.

Answer: [1,-1,0].

2. Every weight is halved, [0.5,-0.5,0]. Same decisions, same mistakes.

3. "great plot" score 1 ok; "awful plot" score 1 wrong ⇒[0,-1,0]; "plot" score 0 ok; "great" score 0 wrong ⇒[1,-1,0]. Nonzero init changes the path.

4. $\mathcal{L} = \max(0, \mathbf{w}^\top f(\mathbf{x}))$ (with the > 0 rule, a score of 0 counts as correct). Gradient 0 if $\mathbf{w}^\top f(\mathbf{x}) \leq 0$; $+f(\mathbf{x})$ if $\mathbf{w}^\top f(\mathbf{x}) > 0$, so the update is $\mathbf{w} - \alpha f(\mathbf{x})$.

5. No. $(1,1)$ and $(-1,-1)$ are negatives of each other, so $\mathbf{w}^\top(1,1) > 0$ forces $\mathbf{w}^\top(-1,-1) < 0$. Both can't be positive.

TOPIC 3

Binary Logistic Regression

Sources: Lecture 1.3.0 Logistic Regression; slides seg-6; Exercise 1.2; A1 Part 2; Fall 2022 Q1(3); Fall 2023 Q1(1).

In plain words

Same score $z = \mathbf{w}^\top f(\mathbf{x})$ as the perceptron, but squash it through the S-shaped logistic (sigmoid) function to get a probability that the label is $+1$. Train by making the observed labels as probable as possible (maximum likelihood), which is the same as minimizing negative log-likelihood with SGD. The update looks like the perceptron's, scaled by "how wrong" the model was: $1 - P(\text{correct label})$. Confident and correct means a tiny update. Confident and wrong means essentially the perceptron update.

3.1 Model

Binary LR (lecture notation, $y \in \{-1, +1\}$)

$$P(y = +1 \mid \mathbf{x}) = \frac{e^{\mathbf{w}^\top f(\mathbf{x})}}{1 + e^{\mathbf{w}^\top f(\mathbf{x})}} = \sigma(\mathbf{w}^\top f(\mathbf{x})), \qquad P(y = -1 \mid \mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^\top f(\mathbf{x})}} = 1 - P(y = +1 \mid \mathbf{x}).$$

Logistic/sigmoid: $\sigma(z) = \dfrac{e^z}{1 + e^z} = \dfrac{1}{1 + e^{-z}}$. Properties: $\sigma(0) = 0.5$, $\sigma(z) \to 1$ as $z \to \infty$, $\to 0$ as $z \to -\infty$, $\sigma(-z) = 1 - \sigma(z)$.
Decision rule: return $+1$ iff $P(y = +1|\mathbf{x}) > 0.5 \iff \mathbf{w}^\top f(\mathbf{x}) > 0$, the same linear boundary as the perceptron. The probabilistic view only matters for *training*.

- **Discriminative** probabilistic model: models $P(y|\mathbf{x})$. A **generative** model models $P(\mathbf{x}, y)$ (Naive Bayes; HMMs in Topic 20).
- "Maximum entropy" (MaxEnt) classifier is another name for logistic regression (lecture 1.4.0).

3.2 Training objective

From likelihood to loss

1. Likelihood of the data: $\max_{\mathbf{w}} \prod_{i=1}^{D} P(y^{(i)} \mid \mathbf{x}^{(i)})$ (discriminative: condition on $\mathbf{x}$).
2. Log is monotonic, so maximize the log-likelihood: $\max_{\mathbf{w}} \sum_{i=1}^{D} \log P(y^{(i)} \mid \mathbf{x}^{(i)})$.
3. Negate to get a loss: $\min_{\mathbf{w}} \sum_{i=1}^{D} -\log P(y^{(i)} \mid \mathbf{x}^{(i)})$, the **negative log-likelihood (NLL)**.
4. Per-example loss $\mathcal{L}_i = -\log P(y^{(i)}|\mathbf{x}^{(i)})$. Optimize with SGD.

3.3 Gradient derivation (be able to reproduce)

Let $z = \mathbf{w}^\top f(\mathbf{x})$. **Case $y = +1$:**

$$\mathcal{L} = -\log \frac{e^z}{1+e^z} = -z + \log(1+e^z)$$

$$\nabla_{\mathbf{w}} \mathcal{L} = -f(\mathbf{x}) + \frac{1}{1+e^z} \cdot e^z \cdot f(\mathbf{x}) \qquad \text{(chain rule; } \nabla_{\mathbf{w}} z = f(\mathbf{x})\text{)}$$

$$= -f(\mathbf{x}) \left(1 - \frac{e^z}{1+e^z}\right) = -f(\mathbf{x}) \left(1 - P(y=+1|\mathbf{x})\right).$$

Case $y = -1$: $\mathcal{L} = -\log \frac{1}{1+e^z} = \log(1+e^z)$, so $\nabla_{\mathbf{w}} \mathcal{L} = f(\mathbf{x})\, P(y=+1|\mathbf{x}) = f(\mathbf{x}) \left(1 - P(y=-1|\mathbf{x})\right)$.

LR SGD updates (Exercise 1.2 form)

$$\begin{aligned} y=+1: &\quad \mathbf{w} \leftarrow \mathbf{w} + \alpha\, f(\mathbf{x}) \left(1 - P(y=+1|\mathbf{x})\right) \\ y=-1: &\quad \mathbf{w} \leftarrow \mathbf{w} - \alpha\, f(\mathbf{x}) \left(1 - P(y=-1|\mathbf{x})\right) \end{aligned} \iff \boxed{\mathbf{w} \leftarrow \mathbf{w} + \alpha\, y\, f(\mathbf{x}) \left(1 - P(y|\mathbf{x})\right)}$$

0/1-label form (A1 code): with $y \in \{0,1\}$ and $p = P(y=1|\mathbf{x})$: $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(y-p)\, f(\mathbf{x})$. Same update: for $y=1$, $y-p = 1-p$; for $y=0$, $y-p = -p = -(1-P(y=0|\mathbf{x}))$.

3.3.1 Relation to the perceptron

- If $P(\text{correct label}) \approx 1$: factor ≈ 0, so (almost) no update. The perceptron also skips correct examples.
- If $P(\text{correct label}) \approx 0$ (confidently wrong): factor ≈ 1, so the update is $\pm\alpha f(\mathbf{x})$, the perceptron update.
- **Exercise 1.2:** the update is numerically closest to the perceptron when $P(y{=}+1|\mathbf{x})$ or $P(y{=}-1|\mathbf{x})$ is close to 1 (that is, close to 0 or 1, not 0.5).
- As a function of z (for $y=+1$), the LR loss $\log(1+e^z) - z = \log(1+e^{-z})$ is a smooth version of the perceptron's hinge. It approaches 0 for large positive z and $\approx -z$ for large negative z. Unlike the perceptron, LR updates on *every* example, even correct ones.
- Lecture conclusion: the two behave almost identically; use whichever is convenient.

Unified loss in terms of the margin yz

Perceptron: $\max(0, -yz)$. Logistic: $\log(1+e^{-yz})$. Both are convex in $\mathbf{w}$. LR with BoW features is a **convex** optimization problem (Fall 2023 Q1(1) II true), so SGD with a suitable step size finds the global optimum.

3.4 Worked numeric example

Worked example: one LR step each for a positive and a negative example

Features (last is the bias feature): $f(\mathbf{x}) = [1, 0, 1, 1]$, $\mathbf{w} = [1, -2, 0.5, -1]$, $y = +1$, $\alpha = 0.1$.

1. $z = 1(1) + 0(-2) + 1(0.5) + 1(-1) = 0.5$.
2. $P(y{=}+1|\mathbf{x}) = \sigma(0.5) = 1/(1+e^{-0.5}) = 0.62246$.
3. Loss $= -\ln 0.62246 = 0.47408$.

4. Gradient $= -(1 - 0.62246) f(\mathbf{x}) = -0.37754\, f(\mathbf{x})$.
5. Update: $\mathbf{w} \leftarrow \mathbf{w} + 0.1(0.37754) f(\mathbf{x}) = [\,1.03775,\ -2,\ 0.53775,\ -0.96225\,]$.

Next example: $f(\mathbf{x}) = [\,0, 1, 1, 1\,]$, $y = -1$, using the new $\mathbf{w}$.

1. $z = -2 + 0.53775 - 0.96225 = -2.42449$, so $P(+1|\mathbf{x}) = \sigma(-2.42449) = 0.08132$ and $P(-1|\mathbf{x}) = 0.91868$.
2. Loss $= -\ln 0.91868 = 0.08482$ (small: already confident and correct).
3. Update: $\mathbf{w} \leftarrow \mathbf{w} - 0.1(1 - 0.91868) f(\mathbf{x}) = \mathbf{w} - 0.008132\, f(\mathbf{x}) = [\,1.03775,\ -2.00813,\ 0.52962,\ -0.97038\,]$.

Only the features that fire ($f_j \neq 0$) change.

Calculator recipe: sigmoid, log-loss, update

1. Compute $z = \sum_j w_j f_j$ (only nonzero f_j).
2. $p = 1 \div (1 + e^{-z})$: key `e^(-z)`, add 1, take reciprocal.
3. Loss for $y = +1$: $-\ln p$; for $y = -1$: $-\ln(1 - p)$. Use `ln` (natural log) unless told otherwise.
4. Scale factor $s = \alpha(1 - p)$ for $y = +1$, or $-\alpha p$ for $y = -1$. New $w_j = w_j + s\, f_j$.
5. Keep 5+ significant digits until the final step, then round as instructed.

3.5 Past exam / exercise problems

Midterm Fall 2022 Q1(3)

A graph of an S-curve from 0 to 1.

Official answer: B: a sigmoid; as $x \to -\infty$ it goes to 0, as $x \to +\infty$ it goes to 1. Distractors: tanh ranges over $(-1, 1)$, not $(0, 1)$; a curve going $1 \to 0$ would be $\sigma(-x)$.

Canvas Exercise 1.2

When is the LR update most similar to the perceptron's?

Answer: When $P(y{=}-1|\mathbf{x}) \approx 1$ or $P(y{=}+1|\mathbf{x}) \approx 1$. The factor $(1 - P)$ is then ≈ 0 or ≈ 1. Exactly 0 or 1 recovers "$\pm\alpha f(\mathbf{x})$ on mistakes, nothing otherwise".

Traps

- The gradient sign: the SGD step *subtracts* the gradient. For $y = +1$ the gradient is $-f(\mathbf{x})(1 - p)$, so you *add* $\alpha f(\mathbf{x})(1 - p)$.
- For $y = -1$ use $1 - P(y{=}-1|\mathbf{x}) = P(y{=}+1|\mathbf{x})$. Don't use $1 - P(+1)$.
- The LR decision threshold is 0.5 on the probability, i.e. 0 on the score. Changing the decision threshold doesn't change training.
- Logistic regression is linear: it cannot solve XOR/"not good vs. not bad" with unigram features, just like the perceptron.
- Use natural log unless told otherwise; NLL in PyTorch uses ln.

3.6 Practice

Practice

1. $\mathbf{w} = [2, -1]$, $f(\mathbf{x}) = [1, 3]$. Compute $P(y{=}+1|\mathbf{x})$, the predicted label, and the NLL if the true $y = -1$.
2. With $\alpha = 0.5$, give the updated $\mathbf{w}$ for Q1.
3. Show that $P(y|\mathbf{x}) = \sigma(y\,\mathbf{w}^\top f(\mathbf{x}))$ for $y \in \{-1, +1\}$.
4. An example has $P(\text{correct}) = 0.99$. Roughly what fraction of the perceptron-sized step does LR take?
5. Why does LR still update on correctly classified examples, and why can that be useful?

Answers

1. $z = 2 - 3 = -1$, $p = \sigma(-1) = 0.26894$, predict -1; NLL $= -\ln(1 - 0.26894) = -\ln 0.73106 = 0.31326$.
2. $\mathbf{w} \leftarrow \mathbf{w} - 0.5(0.26894)[1, 3] = [1.86553,\ -1.40341]$.
3. $y = +1$: $\sigma(z)$. $y = -1$: $\sigma(-z) = 1/(1 + e^z) = P(y{=}-1|\mathbf{x})$.
4. $1 - 0.99 = 0.01$, about 1%.
5. The loss is never exactly 0, so the gradient is nonzero. It keeps pushing toward larger margins, i.e. more confident correct predictions.

TOPIC 7

Optimization: SGD, Step Size, Initialization

Sources: Lectures 1.1.2 Basics of Learning, Gradient Descent, 1.4.1 Optimization Basics, 2.2.1 Neural Net Implementation, 2.2.2 Neural Net Training, Optimization; slides seg-3, seg-8, seg-15, seg-16; Exercises 2.3a, 2.3b; A2 Part 1 (Q1); Fall 2023 Q1(1).

In plain words

Training means searching for weights that make the total loss small. SGD repeatedly picks one example (or a batch), computes which direction makes the loss go *up* (the gradient), and takes a step of size α the other way. Too small a step is slow. Too large a step overshoots, oscillates or diverges. For linear models the loss is convex, so where you start doesn't matter. For neural networks it's non-convex, so the starting point (initialization) matters a lot. In particular, all-zero initialization breaks learning.

7.1 The optimization problem

Objective and SGD (lecture)

$$\min_{\mathbf{w}} \ \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{D} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}) \qquad \text{(training data fixed; } \mathbf{w} \text{ varies)}$$

SGD: repeatedly pick an example i and set $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w})$.
The objective is a *sum over examples* (linear over examples). That's why one example's gradient is a cheap, noisy estimate of the full gradient.

- **Batch/full gradient descent:** sum the gradient over all D examples, then take one step. **Minibatch SGD:** average or sum over a batch (Topic 9).
- **Instances:** perceptron (perceptron loss), logistic regression (NLL), neural networks (NLL through backprop).
- **Convexity:** perceptron loss and LR's NLL are convex in $\mathbf{w}$ (with fixed features). Neural network objectives are **non-convex**.

7.2 Step size: the quadratic example

Worked example: lecture 1.4.1: $\mathcal{L}(w) = w^2$, start at $w = -1$

Gradient $\nabla \mathcal{L} = 2w$, so the update is $w \leftarrow w - \alpha \cdot 2w = (1 - 2\alpha)\, w$.

α	sequence $w_0, w_1, \ldots$	behaviour
0.1	$-1, -0.8, -0.64, -0.512, \ldots$	slow, monotone convergence
0.25	$-1, -0.5, -0.25, -0.125, \ldots$	faster
0.5	$-1,\ \mathbf{0}, 0, 0$	**one step to the optimum**
0.75	$-1, 0.5, -0.25, 0.125, \ldots$	oscillating, still converging
1	$-1, 1, -1, 1, \ldots$	**oscillates forever** (lecture)
1.1	$-1, 1.2, -1.44, 1.728, \ldots$	diverges

General rule for this function (derived): converges iff $|1 - 2\alpha| < 1 \iff 0 < \alpha < 1$; monotone for $\alpha < \frac{1}{2}$; exact in one step at $\alpha = \frac{1}{2}$.

Worked example: Assignment 2 Q1: $f(x_1, x_2) = (x_1 - 1)^2 + 8(x_2 - 1)^2$, start at the origin

1. Gradient: $\nabla f = \big(2(x_1 - 1),\ 16(x_2 - 1)\big)$. Optimum $(1, 1)$.
2. Per-coordinate error after one step: $(x_1 - 1) \leftarrow (1 - 2\alpha)(x_1 - 1)$ and $(x_2 - 1) \leftarrow (1 - 16\alpha)(x_2 - 1)$.
3. Convergence needs *both* $|1 - 2\alpha| < 1$ and $|1 - 16\alpha| < 1$, so $0 < \alpha < \frac{1}{8}$. The steep coordinate (x_2) sets the tipping point: at $\alpha = 1/8$, x_2 oscillates; above it, SGD diverges (e.g. `-lr 1`).
4. Distance after k steps: $\sqrt{(1 - 2\alpha)^{2k} + (1 - 16\alpha)^{2k}}$. One choice is $\alpha \approx 0.1066$, which reaches distance $0.0959 < 0.1$ in **10** steps. No constant α does it in 9 (the best possible after 9 steps is 0.1233).
5. Lesson: **one global step size has to suit every direction**. A flat direction wants a large step and a steep direction a small one. This motivates adaptive methods and Newton's method.

7.3 Choosing step sizes (lecture recipe)

Practical step-size choices

1. **Try them out** over orders of magnitude: $1, 10^{-1}, 10^{-2}, \ldots$ Neural models often need small ones (10^{-4}, 10^{-5}).
2. **Large → small schedules:** $\alpha_t \propto 1/t$ or $1/\sqrt{t}$ for epoch t (fixed schedule).
3. **Decrease on plateau:** reduce α when held-out (dev) performance stagnates.

Second-order and adaptive methods

Newton's method: $\mathbf{w} \leftarrow \mathbf{w} - \left(\nabla^2_{\mathbf{w}} \mathcal{L}\right)^{-1} \nabla_{\mathbf{w}} \mathcal{L}$, using the inverse Hessian (curvature). On a quadratic it jumps straight to the optimum. But the Hessian is **quadratic in the number of features**, so it's infeasible for big models.

Adagrad, Adadelta, Adam: "adaptive" methods, cheap *approximations* to the inverse Hessian that are **linear** in the number of parameters. They rescale the step per coordinate. A rare feature (someone's name) and a common one (*said*) shouldn't share one step size.

Adam (Kingma & Ba 2015) = adaptive per-coordinate step size + **momentum** (an exponentially weighted average of past gradients). It is the default for NLP in this course. Wilson et al. (NeurIPS 2017): adaptive methods can fit training data well but generalize worse at test time than carefully tuned SGD.

7.4 Regularization (lecture: "don't really use")

Classical statistics says don't fully optimize the training loss (bias–variance tradeoff), and add explicit regularization. In this course we usually don't add a regularizer. Instead we rely on **early stopping**, **imperfect optimization**, and tricks like **dropout**.

7.5 Initialization (neural networks)

Why initialization matters

1. **Can't use zeros for parameters that produce hidden layers.** With $\mathbf{z} = g(Vf(\mathbf{x}))$ and $V = 0$, $\mathbf{z} = g(\mathbf{0}) = \mathbf{0}$ for tanh and ReLU. The output layer sees all zeros, every hidden value and gradient is 0, and nothing ever changes. Nonzero random values also **break symmetry** between hidden units.
2. **Don't initialize too large:** tanh/sigmoid units **saturate**. Inputs like ± 5 sit on the flat part of the curve, so the gradient passed back is tiny and learning stalls. ReLU has an analogous problem: if a unit's input is always negative, it outputs 0 with gradient 0 (it's "stuck").
3. **Glorot/Xavier initializer:** $U\left[-\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}, +\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}\right]$ (fan-in/fan-out = number of inputs/outputs of the layer). Goal: keep the variance of activations and gradients about the same across layers, so one step size works everywhere.
4. **Batch normalization** (Ioffe & Szegedy 2015): during training, shift and rescale each layer's activations to mean 0, variance 1 over the batch. Useful for very deep nets, mostly in vision.

Linear models vs. NNs: zero initialization is fine for perceptron/LR (convex, every weight gets its own gradient from the features). For NNs it's fatal, and different random initializations can converge to different solutions.

Canvas Exercise 2.3a

Q1: Network $\mathbf{y} = \text{softmax}(Wg(Vf(\mathbf{x})))$ with all layers initialized to zero.

Answer: It won't learn; the gradients are all 0. Q2: What can improve performance or learning speed?

Answer: Adam (momentum-based) instead of SGD; Xavier initialization; batching. *Not* very large random initial values (saturation).

Canvas Exercise 2.3b (XOR with the PyTorch FFNN example)

Q1: Run 5–10 times on the XOR data.

Answer: Sometimes fits, inconsistent across random initializations (non-convex). Q2: Remove the example $[1, 1]$.

Answer: Fits perfectly (the remaining 3 points are linearly separable, an easy problem). Q3: Zero initialization instead of Xavier.

Answer: Always fails to fit.

Midterm Fall 2023 Q1(1), revisited

"LR with BoW features is convex" is true. "SGD can optimize LR" is true. "The perceptron is SGD on some loss" is true. Neural networks are non-convex, which is why initialization matters for them but not for LR.

Calculator recipe: gradient-descent steps on a quadratic

For $f(\mathbf{x}) = \sum_j a_j(x_j - c_j)^2$: error in coordinate j multiplies by $(1 - 2a_j\alpha)$ each step. After k steps the error is $(1 - 2a_j\alpha)^k\times$ the initial error. Converges iff $\alpha < 1/a_{\max}$.

Traps

- The SGD update *subtracts* the gradient (it minimizes loss). If a question maximizes log-likelihood, gradient *ascent* adds it. Same step.
- "Oscillates" (stays bounded, $|1 - 2a\alpha| = 1$) is different from "diverges" ($|1 - 2a\alpha| > 1$).
- Zero initialization: fatal for hidden layers, harmless for linear models. Large initialization: saturation.
- Newton's method is quadratic in the number of features; Adam etc. are linear.
- With α constant and $\mathbf{w}_0 = 0$, the perceptron is α-invariant. LR and NNs are not.

7.6 Practice

Practice

1. $\mathcal{L}(w) = 3w^2$, $w_0 = 2$. Give w_1, w_2 for $\alpha = 0.1$. Which α reaches 0 in one step? For which α does it diverge?
2. $f(x_1, x_2) = x_1^2 + 4x_2^2$, start $(1, 1)$, $\alpha = 0.2$. Two steps of GD. Converging?
3. A hidden layer uses ReLU and every weight is initialized to -10 with nonnegative BoW inputs. What happens?
4. Name the two ideas Adam combines. Which method uses the inverse Hessian, and why isn't it used for big networks?

Answers

1. $w \leftarrow (1 - 6\alpha)w$: $w_1 = 0.8$, $w_2 = 0.32$. One step at $\alpha = 1/6$. Diverges for $\alpha > 1/3$ (oscillates forever at exactly $1/3$).
2. $x_1 \leftarrow (1 - 0.4)x_1 = 0.6x_1$, $x_2 \leftarrow (1 - 1.6)x_2 = -0.6x_2$: $(0.6, -0.6)$, then $(0.36, 0.36)$. Converging, with x_2 oscillating in sign.
3. Every pre-activation is ≤ 0, so ReLU outputs 0 with zero gradient. The units are dead and nothing learns through them.
4. Adaptive per-coordinate step size + momentum. Newton's method; the Hessian is quadratic in the number of parameters.

MODULE 2

Multiclass and Neural Classification

TOPIC 5

Multiclass Classification: Weights and Features

Sources: Lectures 2.0.0 Multiclass Classification, 2.0.2 Multiclass Classification Examples; slides seg-9, seg-11; Exercise 2.2; Fall 2023 Q1(2).

In plain words

With more than two labels, we can't just ask "is the score above 0?". Instead we give every class its own score and predict the class with the **highest** score (argmax). There are two equivalent bookkeeping schemes. Either each class has its *own weight vector* ("different weights", DW, used for neural nets), or there is one big weight vector and the *features depend on the hypothesized class* ("different features", DF, used for structured prediction).

5.1 Setup

- Label set (output space) $\mathcal{Y}$, e.g. {health, sports, science}.
- **One-vs-all** (one binary classifier per class) is a possible hack, but it can fail. With 3 classes in a line (1s, then 2s, then 3s), class 2 can't be separated from {1,3} linearly. So we use genuinely multiclass models instead.

Two equivalent formulations

Different weights (DW): one weight vector per class.

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \ \mathbf{w}_y^\top f(\mathbf{x}) \qquad (f(\mathbf{x}) \in \mathbb{R}^n, \ \ |\mathcal{Y}| \text{ weight vectors } \mathbf{w}_y \in \mathbb{R}^n)$$

Different features (DF): one weight vector; features conjoin the input with a *hypothesized* label y (not the gold label):

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \ \mathbf{w}^\top f(\mathbf{x}, y) \qquad (f(\mathbf{x}, y) \in \mathbb{R}^{n|\mathcal{Y}|})$$

Standard construction: $f(\mathbf{x}, y)$ is $f(\mathbf{x})$ copied into the block for class y, with zeros elsewhere. Each feature then means "sentence contains word j **and** the label is y". $\mathbf{w}$ is all the $\mathbf{w}_y$ concatenated, and $\mathbf{w}^\top f(\mathbf{x}, y) = \mathbf{w}_y^\top f(\mathbf{x})$.

- DW is natural for neural nets: compute a representation of $\mathbf{x}$ once, then score all classes at the output layer. DF would require rerunning the network for every y.
- DF is essential for structured prediction (tag sequences, trees) where $\mathcal{Y}$ is huge and features must look at properties of both input and output (Topic 20).
- Stack the DW vectors as rows of a matrix $W \in \mathbb{R}^{|\mathcal{Y}| \times n}$: then all scores are $Wf(\mathbf{x})$ (Topic 9).

Worked example: topic classification (lecture)

$\mathbf{x}$ = "too many drug trials, too few patients". Features [drug, patients, baseball]: $f(\mathbf{x}) = [1, 1, 0]$. **DW:** Suppose (made-up) $\mathbf{w}_{\text{health}}^\top f(\mathbf{x}) = 7.6$ and $\mathbf{w}_{\text{sports}}^\top f(\mathbf{x}) = -1.9$, plus a science score. Predict the argmax, **health**.

DF: $f(\mathbf{x}, \text{health}) = [1,1,0\,|\,0,0,0\,|\,0,0,0]$, $f(\mathbf{x}, \text{sports}) = [0,0,0\,|\,1,1,0\,|\,0,0,0]$, $f(\mathbf{x}, \text{science}) = [0,0,0\,|\,0,0,0\,|\,1,1,0]$. With $\mathbf{w} = [\,\mathbf{w}_{\text{health}}\,|\,\mathbf{w}_{\text{sports}}\,|\,\mathbf{w}_{\text{science}}\,]$ the scores are identical to DW.

5.2 Example multiclass tasks (lecture 2.0.2)

- **Text/topic classification** (news, newsgroups): a common benchmark, often artificial.
- **Textual entailment (NLI):** 3 classes for a premise/hypothesis pair: *entailment*, *contradiction*, *neutral*. Hard for BoW; little progress until neural nets.
- **Entity linking/disambiguation:** map the mention "Armstrong" to a Wikipedia article (about 4.5M classes). Needs features that relate the mention and its context to the candidate article.
- **Authorship attribution:** stop-word frequencies and style matter. Twitter study (Schwartz et al.): SVM with character 4-grams + word 2–5-grams, about 70% accuracy on 50 authors with 1,000 tweets each. Distinctive "k-signatures" (n-grams unique to an author).

Midterm Fall 2023 Q1(2)

Input text $\mathbf{x}$, output one of five labels. Most correct name?

Official answer: A. Classification. (Not language modeling, clustering, or parsing.)

Canvas Exercise 2.2

Q1 binary sentiment: all of binary perceptron, binary LR, multiclass perceptron, multiclass LR can be used. Binary is a special case of multiclass. Q2 five-way fine-grained sentiment with a single model:

Answer: multiclass perceptron, multiclass LR (binary methods can't). Q3 want examples "more correct" (large margin, not just barely right):

Answer: logistic regression (keeps updating). Q4 want a concrete "end goal" where training stops updating once the data are separated:

Answer: perceptron.

Traps

- In DF, y in $f(\mathbf{x}, y)$ is the *hypothesized* label being scored, not the gold label. No cheating.
- DF feature dimension $= n \times |\mathcal{Y}|$. The DW parameter count is the same: $|\mathcal{Y}|$ vectors of length n.
- Multiclass prediction is an argmax, not a comparison with 0. Tie-breaking must be specified (Exercise 2.0: ties go to class 3).

5.3 Practice

Practice

1. $f(\mathbf{x}) = [2,0,1]$; $\mathbf{w}_A = [1,1,1]$, $\mathbf{w}_B = [0,3,2]$, $\mathbf{w}_C = [1,-1,0]$. Predicted class?
2. Write $f(\mathbf{x}, B)$ in DF form for Q1 (order A, B, C blocks), and the DF $\mathbf{w}$.
3. 10,000-word vocabulary, 20 topics. How many parameters in DW multiclass LR (no bias)?

Answers

1. Scores A 3, B 2, C 2, so **A**. 2. $f(\mathbf{x}, B) = [\,0,0,0,2,0,1,0,0,0\,]$; $\mathbf{w} = [\,1,1,1,0,3,2,1,-1,0\,]$ (score 2, matching). 3. $20 \times 10{,}000 = 200{,}000$.

TOPIC 6

Multiclass Perceptron (and Multiclass Logistic Regression)

Sources: Lecture 2.0.1 Multiclass Perceptron and Logistic Regression; slides seg-10; Exercises 2.0, 2.1, 2.2.

In plain words

Predict the highest-scoring class. If it's wrong, there are two classes to fix: **push down** the class you wrongly predicted (subtract the features from its weights) and **push up** the correct class (add the features to its weights). The other classes are untouched. Multiclass LR does a "soft" version: every class gets pushed down in proportion to its probability, and the correct class gets pushed up by $(1 - P(\text{correct}))$.

6.1 Multiclass perceptron

Multiclass perceptron (DW form)

for each epoch, **for** each $(\mathbf{x}^{(i)}, y^{(i)})$:

1. $y_{\text{pred}} = \arg\max_{y \in \mathcal{Y}} \mathbf{w}_y^\top f(\mathbf{x}^{(i)})$ (use the stated tie rule).
2. **if** $y_{\text{pred}} \neq y^{(i)}$: $\quad \mathbf{w}_{y_{\text{pred}}} \leftarrow \mathbf{w}_{y_{\text{pred}}} - \alpha f(\mathbf{x}^{(i)}), \qquad \mathbf{w}_{y^{(i)}} \leftarrow \mathbf{w}_{y^{(i)}} + \alpha f(\mathbf{x}^{(i)})$.

DF form: $\mathbf{w} \leftarrow \mathbf{w} + \alpha f(\mathbf{x}^{(i)}, y^{(i)}) - \alpha f(\mathbf{x}^{(i)}, y_{\text{pred}})$. If $y_{\text{pred}} = y^{(i)}$ the two terms cancel, so the "if" is technically unnecessary.

Binary is a special case: with two classes the update pushes $\mathbf{w}_+$ and $\mathbf{w}_-$ in opposite directions. Their difference $\mathbf{w}_+ - \mathbf{w}_-$ follows the binary perceptron (step 2α).

Worked example: lecture / Exercise 2.0 (ties go to class 3 = science)

Features [drug, patients, baseball]. Ex 1: "too many drug trials, too few patients" [1,1,0], gold 1 (health). Ex 2: "baseball players taking drugs" [1,0,1], gold 2 (sports). $\alpha = 1$, all weights 0.

step	scores (h, sp, sc)	pred	gold	weights after: $\mathbf{w}_h$ | $\mathbf{w}_{sp}$ | $\mathbf{w}_{sc}$
ex 1	(0, 0, 0)	3 (tie)	1	[1,1,0] | [0,0,0] | [-1,-1,0]
ex 2	(1, 0, −1)	1	2	[0,1,-1] | [1,0,1] | [-1,-1,0]

Answer: $\mathbf{w}_h = [0, 1, -1]$, $\mathbf{w}_{sp} = [1, 0, 1]$, $\mathbf{w}_{sc} = [-1, -1, 0]$. Exactly two updates, each touching two classes.

6.2 Multiclass logistic regression (softmax regression)

Model

$$P(y = \hat{y} \mid \mathbf{x}) = \frac{\exp\left(\mathbf{w}^\top f(\mathbf{x}, \hat{y})\right)}{\sum_{y' \in \mathcal{Y}} \exp\left(\mathbf{w}^\top f(\mathbf{x}, y')\right)} \qquad \text{(DW: } \exp(\mathbf{w}_{\hat{y}}^\top f(\mathbf{x})) / \textstyle\sum_{y'} \exp(\mathbf{w}_{y'}^\top f(\mathbf{x}))\text{)}$$

softmax: exponentiate the scores, then normalize by their sum. In matrix form, $P(\mathbf{y}|\mathbf{x}) = \text{softmax}(Wf(\mathbf{x}))$.
Binary LR is the 2-class case where the negative class has an implicit zero feature vector: its score is $e^{\mathbf{w}^\top \mathbf{0}} = 1$, which gives $e^z/(1+e^z)$.

Training: NLL and gradient

Loss $\mathcal{L} = -\log P(y^{(i)}|\mathbf{x}^{(i)}) = -\mathbf{w}^\top f(\mathbf{x}, y^{(i)}) + \log \sum_{y'} \exp(\mathbf{w}^\top f(\mathbf{x}, y'))$.

$$\nabla_{\mathbf{w}} \mathcal{L} = -f(\mathbf{x}, y^{(i)}) + \sum_{y'} P(y'|\mathbf{x})\, f(\mathbf{x}, y') \quad \Longleftrightarrow \quad \text{DW: } \nabla_{\mathbf{w}_y} \mathcal{L} = \left(P(y|\mathbf{x}) - \mathbb{1}[y = y^{(i)}]\right) f(\mathbf{x}).$$

SGD ($\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla$): the gold class gets $+\alpha(1 - P(y^{(i)}|\mathbf{x}))f(\mathbf{x})$; every other class y gets $-\alpha P(y|\mathbf{x})f(\mathbf{x})$.

Intuition (lecture): if $P(y^{(i)}|\mathbf{x}) \approx 1$ the gradient is ≈ 0. If all mass is on one wrong class y_{bad}, the update is $+\alpha f(\mathbf{x}, y^{(i)}) - \alpha f(\mathbf{x}, y_{\text{bad}})$, exactly the multiclass perceptron. In between it's a soft version, kicking every class down a bit.

Worked example: Exercise 2.1 (multiclass LR, step size 1)

$f(\mathbf{x}) = [1, 1, 0]$, gold class 1, all weights 0.

1. Scores all 0, so $P = \text{softmax}(0, 0, 0) = (1/3, 1/3, 1/3)$.
2. Gold: $\mathbf{w}_1 \leftarrow \mathbf{w}_1 + (1 - \frac{1}{3})f(\mathbf{x}) = [2/3, 2/3, 0]$. Others: $\mathbf{w}_2, \mathbf{w}_3 \leftarrow -\frac{1}{3}f(\mathbf{x}) = [-1/3, -1/3, 0]$.
3. Q3: training on this example repeatedly, **LR keeps updating** (probability never reaches exactly 1) while the **perceptron stops** once it's correct.

Check: the updates across classes sum to zero ($\frac{2}{3} - \frac{1}{3} - \frac{1}{3} = 0$), since $\sum_y (P(y) - \mathbb{1}[y = y^{(i)}]) = 0$.

Calculator recipe: softmax

Scores $s_1..s_k$. (1) Optional: subtract max s for stability (doesn't change the result). (2) e^{s_j} for each. (3) Divide each by the sum. Example: $(2, 1, 0) \to (7.389, 2.718, 1)/11.107 = (0.6652, 0.2447, 0.0900)$.

Traps

- The multiclass perceptron updates **two** weight vectors on a mistake; multiclass LR updates **all** of them on every example.
- Don't update the wrongly predicted class *up*: it's minus for y_{pred} and plus for the gold class.
- Ties: follow the stated rule exactly. At the first step every score is 0.
- The LR update for non-gold classes is $-\alpha P(y|\mathbf{x})f(\mathbf{x})$, not $-\alpha(1 - P)f(\mathbf{x})$.

6.3 Practice

Practice

1. Classes A, B, C; ties go to the alphabetically first class. Features [f1, f2]. Data: [1,0] gold B; [0,1] gold C; [1,1] gold A. One epoch of multiclass perceptron from zeros.
2. $f(\mathbf{x}) = [1, 2]$, $\mathbf{w}_1 = [1, 0]$, $\mathbf{w}_2 = [0, 1]$, $\mathbf{w}_3 = [0, 0]$, gold class 3, $\alpha = 0.5$. Compute P, the NLL, and the multiclass LR update.

Answers

1. Ex1: scores (0,0,0), pred A, wrong: $\mathbf{w}_A = [-1, 0]$, $\mathbf{w}_B = [1, 0]$. Ex2: scores (0,0,0), pred A, wrong: $\mathbf{w}_A = [-1, -1]$, $\mathbf{w}_C = [0, 1]$. Ex3: scores $(-2, 1, 1)$, tie B/C, pred B, wrong: $\mathbf{w}_B = [0, -1]$, $\mathbf{w}_A = [0, 0]$. Final $\mathbf{w}_A = [0, 0]$, $\mathbf{w}_B = [0, -1]$, $\mathbf{w}_C = [0, 1]$.
2. Scores (1, 2, 0): $P = (0.2447, 0.6652, 0.0900)$; NLL $= -\ln 0.0900 = 2.4076$. Updates: $\mathbf{w}_1 \leftarrow \mathbf{w}_1 - 0.5(0.2447)f(\mathbf{x}) = [0.8776, -0.2447]$; $\mathbf{w}_2 \leftarrow \mathbf{w}_2 - 0.5(0.6652)f(\mathbf{x}) = [-0.3326, 0.3348]$; $\mathbf{w}_3 \leftarrow [0, 0] + 0.5(0.9100)f(\mathbf{x}) = [0.4550, 0.9100]$.

TOPIC 8

Feedforward Neural Networks: Definition and Initialization

Sources: Lectures 2.1.0 Neural Networks, 2.1.1 Neural Network Visualization, 2.2.0 Feedforward Neural Networks, Backpropagation, 2.2.2 Neural Net Training, Optimization; slides seg-12, seg-13, seg-14, seg-16; Exercises 1.1 Q3, 2.3a, 2.3b; Fall 2022 Q1(6), Q3(c).

In plain words

A linear classifier draws one flat boundary in the original feature space. A feedforward network first **transforms** the features into a new "latent" space: multiply by a matrix (warp), add a bias (shift), and apply a nonlinearity (squash). Then it puts a linear classifier on top. In the transformed space, data that wasn't linearly separable (XOR, good/bad/not good/not bad) can become separable. The transformation is *learned* end to end instead of hand-designing conjunction features.

8.1 Definition

One-hidden-layer FFNN (lecture notation)

$$\mathbf{z} = g(Vf(\mathbf{x})), \qquad P(\mathbf{y} \mid \mathbf{x}) = \text{softmax}(W\mathbf{z}) = \text{softmax}(W\,g(Vf(\mathbf{x})))$$

symbol	shape	meaning
$f(\mathbf{x})$	n	input features (BoW, averaged embeddings, ...)
V	$d \times n$	first-layer weights (warps the space); bias optional ($Vf(\mathbf{x}) + \mathbf{b}$)
g	elementwise	nonlinearity: tanh, ReLU, sigmoid, ...
$\mathbf{z}$	d	hidden units = learned latent features
W	$\lvert\mathcal{Y}\rvert \times d$	output layer; row y = class y's weight vector (DW)
$P(\mathbf{y}\lvert\mathbf{x})$	$\lvert\mathcal{Y}\rvert$	vector of class probabilities

Without the hidden layer, $\text{softmax}(Wf(\mathbf{x}))$ is exactly **multiclass logistic regression** (Topic 6). An FFNN is multiclass LR on learned features $\mathbf{z}$.

8.1.1 Nonlinearities

name	formula	range	note
tanh	$\frac{e^a - e^{-a}}{e^a + e^{-a}}$	$(-1, 1)$	saturates at both ends; $\tanh'(a) = 1 - \tanh^2(a)$
ReLU	$\max(0, a)$	$[0, \infty)$	"hinge"; gradient 0 for $a < 0$, 1 for $a > 0$
sigmoid	$1/(1 + e^{-a})$	$(0, 1)$	$\sigma' = \sigma(1 - \sigma)$

Why a nonlinearity is required: without g, $W(Vf(\mathbf{x})) = (WV)f(\mathbf{x})$ collapses to a single linear map,

which is still a linear classifier. *Which* nonlinearity you use matters much less than using one.

8.2 Why it works: the XOR example (lecture 2.1.0)

Worked example: latent space makes XOR separable

Points (x_1, x_2): $(0,0)-$, $(1,0)+$, $(0,1)+$, $(1,1)-$. Not linearly separable. Take $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $g = \tanh$, so $\mathbf{z} = \big(\tanh x_1,\ \tanh x_2,\ \tanh(x_1 + x_2)\big)$. The lecture idealizes $\tanh(1) \approx \tanh(2) \approx 1$, so $\mathbf{z}$: $(0,0,0)-$, $(1,0,1)+$, $(0,1,1)+$, $(1,1,1)-$. The third unit behaves like an OR gate. Now $\mathbf{w} = [-1, -1, 1.5]$ separates them: scores $0,\ 0.5,\ 0.5,\ -0.5$ (with the > 0 rule, 0 is negative). With true tanh values (0.7616, 0.9640) the scores are $0,\ 0.381,\ 0.381,\ -0.077$, which still separates.

Visualization (2.1.1, Chris Olah): the three stages are *warp* (V), *shift* (bias), *nonlinear squash* (tanh into $[-1,1]^2$). Stacking layers can untangle two interleaved spirals. Deep nets = iterated transformations, all learned by backprop.

8.2.1 Deeper networks

$\mathbf{z}_1 = g(V_1 f(\mathbf{x}))$, $\mathbf{z}_2 = g(V_2 \mathbf{z}_1)$, …, $P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\mathbf{z}_k)$. Each matrix's input dimension must equal the previous layer's output dimension. **Universal approximation:** a network with enough hidden units can represent essentially any function, so any finite dataset without identical inputs that have different labels can be fit (Fall 2022 Q3(c)).

8.3 Parameter counting and shape checking

Counting parameters

Linear layer $\mathbb{R}^a \to \mathbb{R}^b$: ab weights ($+b$ biases if used). Nonlinearities and softmax have no parameters. One hidden layer: $dn + |\mathcal{Y}|d$ (no bias) or $dn + d + |\mathcal{Y}|d + |\mathcal{Y}|$ (with bias).

Worked example: counts

BoW $n = 10{,}000$, $d = 100$, 2 classes: $100 \cdot 10{,}000 + 2 \cdot 100 = 1{,}000{,}200$ weights (1,000,302 with biases). 300-dim input, $d = 200$, 2 classes: $60{,}000 + 400 = 60{,}400$ (60,602 with biases).

Midterm Fall 2022 Q1(6)

Use word embeddings + an FFNN to predict binary sentiment. Best implementation? ([in × out]) A. 300-d emb →[300×2]→ReLU→[2×2]→softmax B. 50-d emb →[50×75]→Tanh→[75×2]→log-softmax C. 300-d →[50×100]… D. 50-d →[50×150]→[150×2]→softmax E. 300-d→[300×200]→Tanh→[200×2]→Tanh→softmax.

Official answer: B.

Why: A: a 2-unit hidden layer is a severe bottleneck. C: dimension mismatch (300 into a 50-input layer). D: no nonlinearity between the linear layers, so it's still linear. E: a tanh *before softmax* squashes the logits to $[-1,1]$, so the model can never put probability near 1 on the right class. B:

dimensions chain, has a nonlinearity, and log-softmax pairs with NLL loss.

Canvas Exercise 1.1 Q3

Perceptron fails on good/bad/not good/not bad with the same features. Fix by changing the model?

Answer: Use a nonlinear model such as an FFNN. Not LR (still linear), not more epochs.

8.4 Initialization

See Topic 7, Section 7.5 for the full treatment. Summary:

1. **Never all zeros** for layers feeding hidden units: $\mathbf{z} = g(\mathbf{0}) = \mathbf{0}$, all gradients 0, nothing learns, and there is no symmetry breaking (Exercise 2.3a Q1, 2.3b Q3).
2. **Not too large:** tanh/sigmoid saturate, so gradients vanish. ReLU units can die if their inputs are always negative.
3. **Xavier/Glorot:** $U[-\sqrt{6/(\text{fan-in} + \text{fan-out})}, +\sqrt{6/(\text{fan-in} + \text{fan-out})}]$ keeps the variance of activations and gradients similar across layers.
4. Non-convex, so different random seeds can reach different solutions (XOR sometimes fails, Exercise 2.3b Q1).

Traps

- V is $d \times n$ (output $\times$ input) in lecture math. PyTorch's `nn.Linear(in, out)` lists input first. Exam questions write [in $\times$ out].
- No nonlinearity means a linear model, however many layers.
- Don't put tanh/sigmoid right before the softmax (Fall 2022 Q1(6) E).
- Softmax output with an NLL loss needs a log; log-softmax + NLLLoss is equivalent to raw logits + CrossEntropyLoss.
- Biases: say whether you count them in parameter totals.

8.5 Practice

Practice

1. Input 50-d averaged embeddings, hidden layers of 64 and 32 (ReLU), 5 classes. Give all weight shapes in lecture (out $\times$ in) form and the parameter count with biases.
2. True/False: a 3-layer network with identity activations can solve XOR.
3. With ReLU instead of tanh in the XOR example (same V, no bias), compute $\mathbf{z}$ for the four points. Is there a separating $\mathbf{w}$? If not, fix the third hidden unit with a bias.

Answers

1. $V_1 : 64 \times 50$, $V_2 : 32 \times 64$, $W : 5 \times 32$. Params $= (3200 + 64) + (2048 + 32) + (160 + 5) = 5{,}509$.
2. False: it collapses to one linear map.
3. $\mathbf{z}$: $(0,0,0)$, $(1,0,1)$, $(0,1,1)$, $(1,1,2)$. **Not separable**: on nonnegative inputs ReLU is the identity, so $z_3 = z_1 + z_2$ and the network is still linear here. Fix: $z_3 = \text{ReLU}(x_1 + x_2 - 1)$ gives

$(0,0,0), (1,0,0), (0,1,0), (1,1,1)$, and $\mathbf{w} = [1, 1, -2]$ scores $0, 1, 1, 0$, which is correct under the > 0 rule. The nonlinearity must actually "bend" somewhere the data lives.

TOPIC 9

Training Neural Networks

Sources: Lectures 2.2.0 Feedforward Neural Networks, Backpropagation, 2.2.1 Neural Net Implementation, 2.2.2 Neural Net Training, Optimization; slides seg-14, seg-15, seg-16; Exercises 2.3a/b; A2 Part 2.

In plain words

Same recipe as logistic regression: maximize the log-probability of the correct labels with (stochastic) gradient descent. The new part is getting gradients for the inner layers. **Backpropagation** is the chain rule applied backwards: compute the output error, pass an "error signal" back to the hidden layer, and use it to get gradients for V. In practice PyTorch's automatic differentiation does this for you. You write the forward pass, call `loss.backward()`, then `optimizer.step()`.

9.1 Objective

NLL for an FFNN (lecture form)

Let i^* be the index of the gold label and $\mathbf{e}_{i^*}$ the one-hot selector vector (e.g. $\mathbf{e}_2 = [0,1,0]$).

$$\log P(y = i^* | \mathbf{x}) = \mathbf{e}_{i^*} \cdot \log \text{softmax}(W\mathbf{z}) = \mathbf{w}_{i^*}^\top \mathbf{z} - \log \sum_j \exp(\mathbf{w}_j^\top \mathbf{z}), \qquad \mathcal{L} = -\log P(y = i^* | \mathbf{x}).$$

This is the multiclass LR objective with features $\mathbf{z}$, so the gradient for W is the multiclass LR gradient with $\mathbf{z}$ in place of $f(\mathbf{x})$.

9.2 Backpropagation

Gradients for $\mathbf{z} = g(\mathbf{a})$, $\mathbf{a} = Vf(\mathbf{x})$, $\mathbf{p} = \text{softmax}(W\mathbf{z})$

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial W} &= (\mathbf{p} - \mathbf{e}_{i^*})\,\mathbf{z}^\top && \text{(output layer = multiclass LR)} \\
\delta_z = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} &= W^\top (\mathbf{p} - \mathbf{e}_{i^*}) && \text{("error signal" passed back)} \\
\frac{\partial \mathcal{L}}{\partial \mathbf{a}} &= \delta_z \odot g'(\mathbf{a}) && (\tanh' = 1 - \mathbf{z}^2;\ \text{ReLU}' = \mathbb{1}[a > 0]) \\
\frac{\partial \mathcal{L}}{\partial V} &= \frac{\partial \mathcal{L}}{\partial \mathbf{a}}\, f(\mathbf{x})^\top && \text{(chain rule: } \partial\mathcal{L}/\partial\mathbf{z} \cdot \partial\mathbf{z}/\partial V)
\end{aligned}$$

Pattern (lecture): each parameter's gradient = (error signal coming back from above) $\times$ (input to that layer, $f(\mathbf{x})$ or $\mathbf{z}$), saved from the forward pass. The lecture sketches this; you aren't expected to derive deep backprop, but this structure is the whole idea.

Worked example: one forward/backward/SGD step (verified by finite differences)

$f(\mathbf{x}) = [1,0,1]$, $V = \begin{bmatrix} 0.5 & -1 & 0.2 \\ 0 & 1 & -1 \end{bmatrix}$, $g = \tanh$, $W = \begin{bmatrix} 1 & -1 \\ 0.5 & 2 \end{bmatrix}$, gold class 1, $\alpha = 0.1$.

1. $\mathbf{a} = Vf(\mathbf{x}) = [0.7, -1.0]$, $\mathbf{z} = \tanh \mathbf{a} = [0.60437, -0.76159]$.
2. Scores $W\mathbf{z} = [1.36596, -1.22100]$, $\mathbf{p} = [0.93002, 0.06998]$, $\mathcal{L} = -\ln 0.93002 = 0.07255$.
3. $\mathbf{p} - \mathbf{e}_1 = [-0.06998, 0.06998]$; $\partial\mathcal{L}/\partial W = \begin{bmatrix} -0.04229 & 0.05330 \\ 0.04229 & -0.05330 \end{bmatrix}$.
4. $\delta_z = W^\top(\mathbf{p} - \mathbf{e}_1) = [-0.03499, 0.20995]$; $\odot(1 - \mathbf{z}^2) = [-0.02221, 0.08817]$.
5. $\partial\mathcal{L}/\partial V = \begin{bmatrix} -0.02221 & 0 & -0.02221 \\ 0.08817 & 0 & 0.08817 \end{bmatrix}$. The middle column is 0 because $f_2 = 0$.
6. After the step $\theta \leftarrow \theta - 0.1\nabla$, the loss drops to 0.07003.

9.3 Implementation in PyTorch (lecture 2.2.1, A2)

```
class FFNN(nn.Module):
    def __init__(self, inp, hid, out):
        super().__init__()
        self.V = nn.Linear(inp, hid); self.g = nn.Tanh()
        self.W = nn.Linear(hid, out); self.softmax = nn.Softmax(dim=0)
        nn.init.xavier_uniform_(self.V.weight)   # nonzero init is critical
    def forward(self, x):
        return self.softmax(self.W(self.g(self.V(x))))

ffnn = FFNN(inp, hid, out)
optimizer = optim.Adam(ffnn.parameters(), lr=lr)
for epoch in range(num_epochs):
    for (inp, gold_label) in data:          # gold_label one-hot
        ffnn.zero_grad()                     # clear cached gradients!
        probs = ffnn.forward(inp)
        loss = torch.neg(torch.log(probs)).dot(gold_label)   # NLL
        loss.backward()                      # autodiff computes all gradients
        optimizer.step()                     # Adam update
    # check dev-set performance each epoch
```

- **Computation graph / autodiff:** write forward with differentiable torch operations, and `backward` is generated automatically. Converting to a raw Python number or numpy mid-network **breaks** gradient tracking.
- `zero_grad()` before each backward. PyTorch *accumulates* gradients in place, so forgetting it is a silent bug (A2 handout calls this the "most unintuitive piece").
- Losses: `NLLLoss` expects **log-probabilities**; `CrossEntropyLoss` expects **raw scores (logits)**. Check what yours expects (A2 handout).
- A2's DAN returns a log-probability distribution, uses NLLLoss + Adam, and checks dev accuracy each epoch.

9.4 Practical training techniques (lecture 2.2.2)

Checklist

- **Batching:** process B examples at once. Inputs become [batch, features] matrices, `forward` is unchanged, and probabilities become [batch, classes]. **Sum (or average) the loss over the batch** into one number. Faster through parallel matrix ops, and it can help optimization. Typical sizes 1–100 (16–32 common on GPU). Variable-length sentences are **padded** (A2: PAD index 0, `padding_idx=0`).
- **Initialization:** nonzero, appropriately scaled (Xavier) (Topic 7).
- **Dropout:** randomly zero parts of the network during training; use the full network at test time. It prevents co-adapted features (e.g. two correlated features with weights $+10$ and -9 fitting one odd example). Think of it as an ensemble of subnetworks. Too high a rate hurts; gains are modest.
- **Optimizer:** Adam (adaptive step + momentum) is the default. Tuned SGD may generalize better (Wilson et al. 2017).
- **Gradient clipping:** cap the gradient magnitude to avoid blow-ups.
- **Monitor:** print training loss each epoch and check dev accuracy to catch overfitting. NaNs usually mean the step size is too large ($\log 0$) (A2 tip).

Canvas Exercise 2.3a Q2

What improves performance or learning speed of $\text{softmax}(Wg(Vf(\mathbf{x})))$?

Answer: Adam, Xavier initialization, batching. Not very large random initial values.

Traps

- Order: `zero_grad` → forward → loss → `backward` → `step`.
- The batched loss must be reduced to a scalar before `backward()`.
- `nn.Softmax(dim=0)` works for a single vector; with batches use `dim=1` (classes axis).
- Dropout is *off* at test time (`model.eval()`).
- The output-layer gradient $(\mathbf{p} - \mathbf{e})\mathbf{z}^\top$ has the same form as multiclass LR's.

9.5 Practice

Practice

1. In the worked example, which entries of $\partial\mathcal{L}/\partial V$ are always 0 for this input, and why?
2. With ReLU instead of tanh in the worked example, what are $\mathbf{z}$ and $\partial\mathcal{L}/\partial\mathbf{a}$'s second entry?
3. A batch of 32 examples, 5 classes, hidden size 100, input 300. Give the shapes of the input, hidden activations, output probabilities and the loss.

Answers

1. Column 2 (multiplies $f_2 = 0$). 2. $\mathbf{a} = [0.7, -1]$, so $\mathbf{z} = [0.7, 0]$. $\text{ReLU}'(-1) = 0$, so the second entry is 0: no gradient flows to row 2 of V. 3. [32, 300], [32, 100], [32, 5], scalar.

MODULE 3

Word Embeddings

TOPIC 10

Word Embeddings: Skip-gram, Properties, Bias

Sources: Lectures 3.0.0 Word Embeddings, 3.1.0 Skip-gram, 3.1.1 Other Word Embedding Methods, 3.1.2 Bias in Word Embeddings; slides seg-17–seg-20; Exercises 3.0, 3.1; Fall 2022 Q1(1), Q1(5), Q1(11–13), Q2(a), Q5; Fall 2023 Q1(4), Q3.

In plain words

One-hot/BoW vectors make every word equally far from every other: *movie* and *film* are orthogonal, so "movie was good" and "film is great" look unrelated. **Word embeddings** are short dense vectors (50–300 dims) where words used in similar contexts get similar vectors. **Distributional hypothesis** (Firth 1957): "You shall know a word by the company it keeps." Skip-gram learns these vectors from raw text, with no labels, by making each word's vector good at predicting the words around it.

10.1 Skip-gram model

Skip-gram (word2vec, Mikolov et al. 2013)

Input: a large raw corpus. **Output:** for each word *type* w, a word vector $\mathbf{v}_w$ and a context vector $\mathbf{c}_w$, both in $\mathbb{R}^d$. **Hyperparameters:** dimension d (50–300) and window size k.
Training pairs: for every token, pair it (as the *word*) with every token up to k positions away (as the *context*).

$$P(\text{context} = y \mid \text{word} = x) = \frac{\exp(\mathbf{v}_x \cdot \mathbf{c}_y)}{\sum_{y' \in V} \exp(\mathbf{v}_x \cdot \mathbf{c}_{y'})}$$

The denominator sums over the **whole vocabulary**. This is multiclass LR where the "features" are $\mathbf{v}_x$ and the "class weights" are the context vectors.
Parameters: $V \in \mathbb{R}^{|V| \times d}$ (word vectors) and $C \in \mathbb{R}^{|V| \times d}$ (context vectors), for $2|V|d$ in total.
Objective: maximize $\sum_{(x,y) \in \text{pairs}} \log P(y|x)$ by SGD from **random** initialization. This is an "impossible" problem: a word has many different contexts, so you can never get every pair to probability 1 (unlike classification).

Matrix view (3.1.1): $P(\text{context} \mid x) = \text{softmax}(W\mathbf{v}_x)$, where row y of W is $\mathbf{c}_y$. The name "skip-gram" comes from predicting words that skip over intermediate words (window $k > 1$).

Exam procedure: skip-gram probabilities

1. Compute $\mathbf{v}_x \cdot \mathbf{c}_y$ for **every** y in the vocabulary, including $y = x$ and words never seen next to x.
2. Exponentiate (use the question's value for e if given, e.g. $e = 3$; $e^0 = 1$).
3. Divide each by the sum.

To find *optimal* vectors: make the dot products with observed contexts large and the rest very negative ($\pm\infty$ in idealized questions). If a word has two equally frequent contexts, the optimum is probability $\frac{1}{2}$ each.

Worked example: lecture / Exercise 3.1 Q1 ("I saw", $e = 3$)

$\mathbf{v}_I = [1,0]$, $\mathbf{v}_{saw} = [0,1]$, $\mathbf{c}_{saw} = [1,0]$, $\mathbf{c}_I = [0,1]$. Pairs (k=1): (I, saw), (saw, I). $P(\cdot \mid saw)$: $\mathbf{v}_{saw} \cdot \mathbf{c}_I = 1 \to e^1 = 3$; $\mathbf{v}_{saw} \cdot \mathbf{c}_{saw} = 0 \to 1$. So $P(I|saw) = 3/4$, $P(saw|saw) = 1/4$. Note $\mathbf{c}_I$ sits on top of $\mathbf{v}_{saw}$: word space and context space are different (roughly rotations of each other). A word's vector is close to its contexts' *context* vectors, not to their word vectors.

Canvas Exercise 3.1 Q2–Q4

Q2: with $\mathbf{v}_I = [1,0]$, $\mathbf{v}_{saw} = [0,1]$, the optimal context vectors are

Answer: $\mathbf{c}_{saw} = [1,-\infty]$, $\mathbf{c}_I = [-\infty,1]$. This gives $P(saw|I) \to 1$ and $P(I|saw) \to 1$.

Q3: corpus "the dog", "the cat", "a dog" with $k = 1$.

Answer: 6 pairs (two per sentence).

Q4: contexts fixed at $\mathbf{c}_a = \mathbf{c}_{the} = (1,0)$, $\mathbf{c}_{cat} = \mathbf{c}_{dog} = (0,1)$. "the" occurs with cat and dog once each, so the best is $P = \frac{1}{2}$ each:

Answer: $\mathbf{v}_{the} = (-\infty, 0)$ or $(0, \infty)$.

Counting skip-gram pairs

A sentence of L tokens with window k (and $L > k$) gives $2\left(kL - \frac{k(k+1)}{2}\right)$ ordered pairs. Each unordered neighbor pair appears in both directions.

Midterm Fall 2022 Q1(1)

All skip-gram (word, context) pairs for *the cat ran away*, window ± 2.

Official answer: C: [the, cat], [the, ran], [cat, the], [cat, ran], [cat, away], [ran, the], [ran, cat], [ran, away], [away, cat], [away, ran]. That's $2(2 \cdot 4 - 3) = 10$ pairs. Pairs are single words, both directions, and nothing farther than 2 away (the/away is excluded).

10.2 Other embedding methods (3.1.1)

method	key idea
CBOW	Flip skip-gram: **sum the context words' embeddings** and predict the center word: $P(w\|\text{ctx}) = \text{softmax}(W(\mathbf{c}_{w_{-1}} + \mathbf{c}_{w_{+1}}))$. Same parameter count; performs similarly.
Hierarchical softmax	Replace the $\|V\|$-way softmax with a binary tree (e.g. Huffman) of binary decisions: $O(\log \|V\|)$ dot products per prediction instead of $O(\|V\|)$. Same number of parameters ($\|V\| - 1$ internal nodes).
Skip-gram w/ negative sampling (SGNS)	Binary classification of (word, context) pairs as real or fake: score $\mathbf{v}_w \cdot \mathbf{c}_c$ through a sigmoid. Real pairs come from data; k negatives by sampling random contexts. Most common in practice (efficient). max $\log \sigma(\mathbf{v}_w \cdot \mathbf{c}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_w \cdot \mathbf{c}_{c_i'})$.
Matrix factorization	Levy & Goldberg (2014): SGNS implicitly factorizes the $\|V\| \times \|V\|$ matrix with entries $\text{PMI}(w, c) - \log k$, where $\text{PMI} = \log \frac{P(w,c)}{P(w)P(c)}$ (negatives from the unigram distribution; weighted factorization).
GloVe	Regression on the **log co-occurrence counts** matrix using word/context vectors plus bias constants. Cost is quadratic in $\|V\|$ but constant in corpus size. Most widely used static embeddings (A2 uses GloVe 50d/300d).
fastText	SGNS where a word = sum of its **character n-gram** vectors (3–6-grams) $\Rightarrow$ can embed unseen words.
Contextual / subword	Pre-trained models (ELMo, BERT) split rare words into subwords and compute *context-dependent* embeddings with RNNs/Transformers (Topic 16, Topic 17).

Midterm Fall 2022 Q2(a)

Difference between skip-gram and CBOW?

Official answer: CBOW uses all surrounding context words at once to predict the current word; skip-gram uses the input word to predict each context word one at a time.

10.3 Properties of embeddings

- Similar words cluster: movie $\approx$ film. This helps a downstream model generalize to words not in its training set ("film is great" after training on "movie was good").
- **Antonyms can be close:** *good*/*bad* share contexts ("good food", "bad food"). Nothing forces opposites to have opposite vectors. This is a known problem for sentiment.
- Words that co-occur but behave differently (*I*/*saw*, *baseball*/*bat*) are *not* necessarily similar. Similarity comes from *substitutability* (sharing contexts), not co-occurrence.
- Words never sharing contexts (*tuck*/*ship*) have mostly unrelated embeddings.
- **Polysemy:** one vector per word type for all senses (*bark* of a tree vs. a dog; *film* reel vs. movie).
- Evaluation: *intrinsic* (word similarity) vs. *extrinsic* (downstream task performance, which is what we care about).

Canvas Exercise 3.0

Q1 true statements:

Answer: similar meanings $\Rightarrow$ similar embeddings; opposite meanings *might* have similar embeddings; words never in the same context have mostly unrelated embeddings. False: "opposites have opposite embeddings"; "dissimilar words that co-occur (baseball/bat) have similar embeddings".

Q2 shortcomings:

Answer: polysemous words get one embedding; embeddings can encode biases. (Tokenization cost is not a real issue.)

10.3.1 Similarity measures

Dot product vs. cosine

$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ vs. $\cos(\mathbf{a}, \mathbf{b}) = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$. The dot product grows with vector length, so long (often frequent) vectors look similar to everything. Cosine ignores length. After normalizing all vectors to length 1, dot product = cosine.

Midterm Fall 2022 Q1(11)–(13)

good $= [0, 0, 3]$, bad $= [0, 1, 3]$, the $= [10, 10, 10]$. Dot products: good·bad $= 9$, good·the $= 30$, bad·the $= 40$, so most similar by dot product:

Official answer: C (bad, the). Cosine: $9/(3\sqrt{10}) = 0.949$; $30/(3 \cdot 17.32) = 0.577$; $40/(3.162 \cdot 17.32) = 0.730$, so

Official answer: A (good, bad). Normalized dot product = cosine:

Official answer: A.

10.4 Bias in word embeddings (3.1.2)

- Web text encodes societal biases, and embeddings absorb them at the very first stage of the pipeline.
- Bolukbasi et al. (2016), "Man is to Computer Programmer as Woman is to Homemaker?": take the *she*−*he* direction (a gender axis). Occupations most aligned with each end reflect stereotypes (*homemaker* at the extreme *she* end). Similar effects show up for race and religion.
- **Debiasing:** identify a gender subspace from gendered pairs (she/he, woman/man, ...). For words that should be neutral (*homemaker*), project onto the subspace and **subtract that projection**: $\mathbf{w}' = \mathbf{w} - \text{proj}_{\text{gender}}(\mathbf{w})$. This "neutralizes" the word along the gender axis.
- **Limitation:** bias is pervasive in the geometry. *homemaker* stays near other words (*child*, *caring*) that still carry gender associations, so neutralizing one direction doesn't remove it. The method also assumes a gender binary. This is an open problem.

Midterm Fall 2022 Q1(5)

What bias do Bolukbasi et al. remove?

Official answer: A: correlation with a he–she axis in embedding space.

Midterm Fall 2023 Q1(4)

True of Bolukbasi debiasing?

Official answer: C: it "neutralizes" words with respect to a gender subspace. Not A (no vocabulary removal), not B (dimensionality isn't reduced), not D ("homemaker" can still be similar to historically gendered concepts).

10.5 Long skip-gram problems

Midterm Fall 2023 Q3 ($e = 3$; $e^{10} \approx \infty$)

$\mathbf{v}_1 = (0,0)$, $\mathbf{v}_2 = (1,1)$, $\mathbf{v}_3 = (-1,1)$; $\mathbf{c}_1 = (0,0)$, $\mathbf{c}_2 = (0,1)$, $\mathbf{c}_3 = (1,0)$.
(a) Distribution for $\mathbf{v}_1$: all dot products 0, so

Official answer: $(1/3, 1/3, 1/3)$.
(b) $\mathbf{v}_2$: dots $(0,1,1) \to (1,3,3)/7 =$

Official answer: $(1/7, 3/7, 3/7)$.
(c) New words 4, 5 co-occur only with each other (4–4, 4–5, 5–5). Minimum extra dimensions while preserving existing relations?

Official answer: 1 (intended).
(d) **Erratum (official):** because $\mathbf{v}_1 = \mathbf{0}$, word 1 always gives a uniform distribution, so the intended "saturation" is impossible. A nearly optimal solution with 0 extra dimensions exists: $\mathbf{v}_4 = \mathbf{v}_5 = \mathbf{c}_4 = \mathbf{c}_5 = (0,-10)$. Then $\mathbf{v}_4 \cdot \mathbf{c}_{4,5} = 100$, which dominates every other dot product. The intended solution used 1 extra dimension with a similar structure, e.g. $\mathbf{v}_{4,5} = \mathbf{c}_{4,5} = (0,0,10)$, existing vectors padded with 0.

Midterm Fall 2022 Q5 (bigram skip-gram)

(a) Learn a vector $\mathbf{v}_b$ for every bigram, with unigram contexts: $P(y|b) = \dfrac{\exp(\mathbf{v}_b \cdot \mathbf{c}_y)}{\sum_{y' \in V} \exp(\mathbf{v}_b \cdot \mathbf{c}_{y'})}$.
(b) Runtime for one pair:

Official answer: $O(|V|d)$ ($|V|$ dot products of length d in the denominator).
(c) Training M examples for N epochs, each step costing E:

Official answer: $O(MNE)$.
(d) Parameters:

Official answer: $|B|d + |V|d$ (bigram vectors + context vectors).
(e) Represent a bigram as the sum of its word vectors: $P(y|b) = \dfrac{\exp((\mathbf{v}_{b_1} + \mathbf{v}_{b_2}) \cdot \mathbf{c}_y)}{\sum_{y'} \exp((\mathbf{v}_{b_1} + \mathbf{v}_{b_2}) \cdot \mathbf{c}_{y'})}$.
(f) Runtime:

Official answer: $O(|V|d)$. (g) Parameters:

Official answer: $2|V|d$, fewer than before since $|B| \gg |V|$.

Traps

- Include **every** vocabulary word in the softmax denominator, including the word itself.
- Use the e value the question gives ($e = 3$) and keep fractions exact.
- Word vectors and context vectors are *different* parameters. $P(y|x)$ uses $\mathbf{v}_x \cdot \mathbf{c}_y$, not $\mathbf{v}_x \cdot \mathbf{v}_y$.
- Skip-gram pairs are ordered and come in both directions; count them carefully at sentence edges.
- Dot product favors long vectors; cosine doesn't.
- The zero vector gives a uniform distribution regardless of the context vectors (Fall 2023 Q3 erratum).

10.6 Practice

Practice

1. Using the Fall 2023 Q3 vectors, compute the context distribution for $\mathbf{v}_3$ ($e = 3$).
2. Window $k = 1$: list all skip-gram pairs for "a b a c". How many are there?
3. Vocabulary 50,000, $d = 300$. How many skip-gram parameters? How many dot products to compute one $P(y|x)$ with full softmax vs. hierarchical softmax (approximately)?
4. $\mathbf{a} = [1, 2]$, $\mathbf{b} = [2, 4]$, $\mathbf{c} = [3, -1]$. Which pair is most similar by cosine? By dot product?

Answers

1. Dots with $\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$: $0, 1, -1 \to 1, 3, \frac{1}{3}$; sum $\frac{13}{3}$; so $(3/13,\ 9/13,\ 1/13)$.
2. (a,b),(b,a),(b,a),(a,b),(a,c),(c,a): 6 pairs $[2(1 \cdot 4 - 1) = 6]$.
3. $2 \times 50{,}000 \times 300 = 30{,}000{,}000$. Full softmax 50,000 dot products; hierarchical $\approx \log_2 50{,}000 \approx 16$.
4. Cosine: (a,b) $= 1$ (parallel). Dot: a·b $= 10$, b·c $= 2$, a·c $= 1$, so (a,b) again.

TOPIC 11

Deep Averaging Networks (Assignment 2)

Sources: Lecture 3.2.0 Applying Embeddings, Deep Averaging Networks; slides seg-21; Assignment 2 (Parts 2–3); Fall 2023 Q1(3), Q1(5–6), Q2(c–e).

In plain words

Look up each word's embedding, **average** them into one sentence vector, and feed that into a feedforward network with a softmax output. Simple and fast, and in 2015 it matched much fancier syntax-based models on sentiment. The cost: averaging throws away word order and interactions between words, so it can't tell "not bad" from "bad, not" and struggles with negation and contrast.

11.1 Model

DAN (Iyyer et al. 2015; A2 Part 2)

For input $s = (w_1, \ldots, w_n)$ with embedding function $e(\cdot)$:

$$\mathbf{a} = \frac{1}{n}\sum_{i=1}^{n} e(w_i) \in \mathbb{R}^d, \qquad \mathbf{h} = g(V\mathbf{a}), \qquad P(\mathbf{y}|s) = \text{softmax}(W\mathbf{h}) \quad \text{(one or more hidden layers)}.$$

Learnable parameters: feedforward layer weights (and biases) and the word embeddings, if not frozen. **No attention**, no W^K, no attention maps (Fall 2023 Q1(3)).

11.1.1 Three ways to use embeddings in the first layer (lecture)

1. **Learn from scratch** (random init + backprop). Works with enough data; can't help with words unseen in training. (The original DAN results used this.)
2. **Initialize with GloVe, keep frozen.** Efficient (no embedding updates), but inflexible if GloVe's geometry doesn't suit the task. Example: *good*/*bad* are close.
3. **Initialize with GloVe, fine-tune.** Backprop into the embedding layer. Often the standard choice for sentiment. Which is best depends on the task and amount of data.

11.2 Assignment 2 implementation details

- Data = A1 sentiment data, **lowercased** (GloVe has only lowercase). Embeddings: GloVe 50d or 300d (300d is graded), "relativized" to the dataset's words.
- `WordEmbeddings` indexer: **PAD = index 0, UNK = index 1**, both mapped to the zero vector by default. `get_initialized_embedding_layer(frozen=...)` returns an `nn.Embedding` initialized with GloVe.
- The module returns **log-probabilities**, trained with `NLLLoss` (or raw scores + `CrossEntropyLoss`), `zero_grad` / `backward` / `step`, Adam.
- **Batching:** pad sentences to equal length with index 0 (`padding_idx=0`). The average must exclude pads (divide by the true length, not the padded length). Your implementation: masked

sum / count of real tokens; dropout 0.2 on inputs, 100-unit ReLU hidden layer, dropout 0.3, 2-way log-softmax; Adam lr 0.001, 10 epochs; dev accuracy 81.5% (required 77%).

- **Part 3, typos** (dev-typo set; typos never in the first 3 characters or in short words): a typo makes a word miss the indexer and become UNK/zero. Option 1: spelling correction by edit distance (fast candidates needed; naive $O(|V|)$ per word is too slow). Option 2: **prefix embeddings** (first 3 characters, initialized from averages of words with that prefix, fine-tuned). Your implementation: edit-distance correction among same-3-prefix candidates, 80.6% on typo dev (required 74%).

Shape / parameter count for a DAN

Input: word indices [batch, len] $\rightarrow$ embeddings [batch, len, d] $\rightarrow$ average over len $\rightarrow$ [batch, d] $\rightarrow$ Linear $d \rightarrow h \rightarrow g \rightarrow$ Linear $h \rightarrow C \rightarrow$ (log-)softmax [batch, C]. Params $= |V|d$ (embeddings, if trained) $+ (dh + h) + (hC + C)$. Example: $|V| = 15{,}000$, $d = 300$, $h = 100$, $C = 2$: $4{,}500{,}000 + 30{,}100 + 202 = 4{,}530{,}302$. If embeddings are frozen, only 30,302 are trainable.

Worked example: DAN forward pass

Embeddings ($d = 2$): good $= [1, 2]$, not $= [-1, 0]$, movie $= [0, 1]$. Sentence "not good movie". $\mathbf{a} = \frac{1}{3}([-1, 0] + [1, 2] + [0, 1]) = [0, 1]$. Hidden: $V = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$, ReLU: $V\mathbf{a} = [1, 1]$, so $\mathbf{h} = [1, 1]$. Output $W = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$ (rows: neg, pos): scores $[0, 1]$, $P = \text{softmax} = [0.269, 0.731]$, so it predicts positive. **Order-blindness:** "good not movie" and "movie not good" give the same $\mathbf{a}$, hence the same prediction.

11.3 Limitations (exam favorite)

1. **Order is lost:** averaging is permutation-invariant, so "not bad" $\equiv$ "bad not". It can't model negation scope or composition. Lecture example: on *good / bad / not good / not bad*, the DAN treats both *not* and *bad* as negative evidence, so it predicts negative for 3 of the 4 phrases. That gets *not bad* wrong.
2. **Mixed sentiment / contrast:** positive and negative words average out. "but" clauses can't override earlier content.
3. **No word-to-word interactions:** the FFNN models interactions among *dimensions of the average*, not between specific words.
4. **Unseen/misspelled words** map to UNK (zero vector) and contribute nothing. Since the average still divides by them, they dilute the signal.
5. **Strengths:** simple, fast, robust to long-range *position* effects. Unlike an RNN, it isn't dominated by the last tokens (Fall 2023 Q1(5–6)).

Historical point: in 2015 people believed sentiment required syntactic (tree-structured) composition. The DAN matched recursive tree networks and Kim's CNN. That didn't show syntax is useless; it showed those models weren't exploiting it much yet.

11.4 Past exam problems

Midterm Fall 2023 Q1(3)

Parameters that can be learned in a DAN? I. feedforward layers; II. W^K; III. word embeddings; IV. attention map weights.

Official answer: I, III.

Midterm Fall 2023 Q1(5)–(6)

A unidirectional RNN predicts very positive for a long negative review ending "At least the dinner was excellent." Why?

Official answer: D: RNN state is heavily influenced by recent tokens. What might work better?

Official answer: I. BoW (more negative words overall), II. DAN, III. Transformer. The DAN averages over all words equally.

A2 / Fall 2023 Q2(c–e) connection

Typos break word lookup. Subword tokenization (Topic 17) or edit-distance repair restores the signal. Fall 2023 judged typo repair less fragile for typos anywhere in a word.

Traps

- The DAN has no attention or recurrence; don't list W^Q, W^K, W^V as DAN parameters.
- With padding, average only over real tokens.
- "Deep" refers to the feedforward layers after averaging; the averaging itself has no parameters.
- Frozen vs. fine-tuned embeddings changes the *trainable* parameter count.

11.5 Practice

Practice

1. Give two sentences with opposite sentiment that a DAN must classify identically.
2. A DAN uses 50-d embeddings (frozen), hidden 64, 3 classes, biases included. How many trainable parameters?
3. Why might padding tokens hurt if you naively average over the padded length?

Answers

1. "the food was good , the service was not bad" vs. "the food was not good , the service was bad" (same multiset of words). 2. $50 \cdot 64 + 64 + 64 \cdot 3 + 3 = 3{,}459$. 3. PAD vectors are zero, so dividing by the padded length shrinks the average toward 0, and the shrinkage depends on how much padding the batch has. The same sentence gets different representations in different batches.

MODULE 4

Language Modeling and Self-Attention

TOPIC 12

n-gram Language Modeling (+ Smoothing, Perplexity, Neural LMs)

Sources: Lectures 4.0.0 n-gram LMs, 4.0.1 Smoothing in n-gram LMs, 4.0.2 LM Evaluation, 4.1.0 Neural Language Models, 4.1.1 RNNs and their Shortcomings; slides seg-41, seg-42, seg-lm3, seg-43, seg-lm5; Exercise 4.0; Fall 2023 Q1(5), Q1(7–8).

In plain words

A language model (LM) assigns a probability to every sentence. By the chain rule, that's the product of each word's probability given all previous words. An **n-gram LM** simplifies by looking only at the previous $n-1$ words. Each conditional probability is then just "how often did this word follow this context in a big corpus?": **count and normalize**. Unseen combinations get probability 0, so we **smooth** by shaving a little mass off seen events and backing off to shorter contexts. LMs are evaluated by **perplexity**: exp of the average negative log-probability on held-out text. Lower is better.

12.1 Definition

Chain rule and the n-gram assumption

$$P(w_1,\dots,w_m) = \prod_{i=1}^{m} P(w_i \mid w_1,\dots,w_{i-1}) \quad \text{(exact)} \qquad \approx \qquad \prod_{i=1}^{m} P(w_i \mid w_{i-n+1},\dots,w_{i-1}) \quad (n\text{-gram}).$$

Bigram ($n=2$): $P(\mathbf{w}) = P(w_1|\langle s\rangle)P(w_2|w_1)P(w_3|w_2)\cdots$ **Trigram**: $P(w_1|\langle s\rangle,\langle s\rangle)P(w_2|\langle s\rangle,w_1)P(w_3|w_1,w_2)\cdots$ ($n-1$ start symbols). An n-gram LM is an $(n-1)$**-order Markov model** ($w_3 \perp w_1 \mid w_2$ in a bigram LM), like HMM tag transitions (Topic 20). Typical n: 3–7 (5-grams common).

Parameters and estimation (count and normalize)

Multinomial distributions: a bigram model has a $|V| \times |V|$ table (one distribution over next words per context word). Maximum-likelihood estimate:

$$P(w \mid \text{ctx}) = \frac{\text{count}(\text{ctx}, w)}{\text{count}(\text{ctx})}, \qquad \text{e.g. } P(\text{dog}|\text{the}) = \frac{c(\text{the dog})}{c(\text{the})}.$$

Distributions are typically *flat* (after *the*, almost anything can follow). Like skip-gram, it's an under-constrained prediction problem. Whether a stop symbol $\langle/s\rangle$ is generated depends on the setup. Follow the question (Fall 2023 Q7 has no stop symbol).

Worked example: bigram estimation and sentence probability

Corpus: "$\langle s\rangle$ I am Sam $\langle/s\rangle$", "$\langle s\rangle$ Sam I am $\langle/s\rangle$", "$\langle s\rangle$ I do not like green eggs and ham $\langle/s\rangle$".

- $P(\text{I}|\langle s\rangle) = 2/3$, $P(\text{Sam}|\langle s\rangle) = 1/3$, $P(\text{am}|\text{I}) = 2/3$, $P(\text{do}|\text{I}) = 1/3$, $P(\text{Sam}|\text{am}) = 1/2$,

$P(\langle/s\rangle|\text{am}) = 1/2$, $P(\langle/s\rangle|\text{Sam}) = 1/2$.

- $P(\langle s\rangle\ \text{I am Sam}\ \langle/s\rangle) = \frac{2}{3}\cdot\frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{2} = \frac{1}{9} \approx 0.111$.
- Perplexity of this sentence over its 4 predictions: $(1/9)^{-1/4} = \sqrt{3} \approx 1.732$.
- $P(\text{I am ham})$: $c(\text{am ham}) = 0$, so the whole product is 0. That's why we smooth.

Midterm Fall 2023 Q1(7)–(8)

Bigram counts (a, b) = times b followed a: (the, frog) 4; (frog, is) 2; (frog, was) 4; (is, UNK) 1; (is, frog) 1; (UNK, UNK) 0. Vocabulary {the, frog, is, was}; everything else maps to UNK; $P(\text{the}|\langle s\rangle) = 1$.
(7) $P(\text{the frog is blue})$, no smoothing: $1\cdot\frac{4}{4}\cdot\frac{2}{6}\cdot\frac{1}{2} =$

Official answer: D. $1/6$ (blue→UNK; frog was followed 6 times; is 2 times).
(8) Which added sentence increases this probability most?

Official answer: A. "the frog is black": $1\cdot\frac{5}{5}\cdot\frac{3}{7}\cdot\frac{2}{3} = \frac{2}{7} \approx 0.286$. Check the others: B "frog is frog is frog is" gives $\frac{5}{9}\cdot\frac{1}{4} = \frac{5}{36} \approx 0.139$; C "the is blue" gives $\frac{4}{5}\cdot\frac{1}{3}\cdot\frac{2}{3} = \frac{8}{45} \approx 0.178$; D "frog frog frog frog" gives $\frac{2}{9}\cdot\frac{1}{2} = \frac{1}{9}$. **Method:** add the new bigram counts, recompute *both* numerators and context totals.

12.2 Uses (lecture)

Phrase-based MT (score how well stitched phrases read in the target language); grammatical error correction (does swapping an article raise the LM probability?); and, generally, LMs as "word2vec++": next-word prediction forces learning rich representations (pre-training, Topic 16).

12.3 Smoothing: absolute discounting

Absolute discounting with backoff (lecture form)

$$P_{AD}(w \mid \text{ctx}) = \frac{\max(c(\text{ctx}, w) - k,\, 0)}{c(\text{ctx})} + \lambda(\text{ctx})\, P_{AD}(w \mid \text{shorter ctx}), \qquad 0 < k < 1,$$

$$\lambda(\text{ctx}) = \frac{k \cdot \#\{\text{distinct word types seen after ctx}\}}{c(\text{ctx})} \quad \text{(exactly the mass removed, so it normalizes).}$$

Recurse down to unigrams, so any word seen in training gets nonzero probability. **Kneser–Ney** (the standard): same framework, but the lower-order distribution uses the *fertility* of contexts (how many distinct words a context is followed by), not raw counts.

Worked example: lecture: "want to go to ___" ($k = 0.2$)

Counts: Maui 2, class 1, campus 1 (total 4, 3 types). Discounted: Maui $1.8/4 = 0.45$, class $0.8/4 = 0.2$, campus 0.2. Reserved $\lambda = 3(0.2)/4 = 0.6/4 = 0.15$. $P(\text{Austin}|\text{want to go to}) = 0 + 0.15 \cdot P_{AD}(\text{Austin}|\text{to go to})$, which is nonzero once the backoff reaches a context where Austin was seen. Why more context helps: "want to go to" vs. "hate to go to" predict very different next words (Austin vs. class), but longer contexts are sparse.

12.4 Evaluation: perplexity

Perplexity (lecture 4.0.2)

Accuracy makes little sense (next-word prediction is "impossible"). Use the log-likelihood of **held-out** data, averaged per token:

$$\text{PPL} = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_{<i})\Big) = \Big(\prod_{i=1}^{N} P(w_i|w_{<i})\Big)^{-1/N}.$$

Lower is better. Interpretation: the average *branching factor*, i.e. the geometric mean of $1/P$. Example: probabilities $\frac{1}{4}, \frac{1}{3}, \frac{1}{4}, \frac{1}{3}$ give PPL $= (4 \cdot 3 \cdot 4 \cdot 3)^{1/4} = \sqrt{12} = 3.464$. A uniform model over $|V|$ words has PPL $= |V|$. Typical values range from about 10 to 200 depending on the domain. The base doesn't matter as long as exp and log match ($2^{-\frac{1}{N}\sum \log_2 p}$ is the same number).

Calculator recipe: perplexity

(1) Get each token's $P(w_i|\text{context})$. (2) Sum ln P. (3) Divide by $-N$ (number of predicted tokens). (4) e^x. Shortcut: PPL $= (\prod p_i)^{-1/N}$. If you have the average NLL (the A3 loss in nats), PPL $= e^{\text{avg NLL}}$.

12.5 Neural LMs and RNNs (background for Topic 13, Topic 15)

- Neural LM: $P(y|w_{1:i-1}) = \text{softmax}\big(\mathbf{u}_y \cdot f(w_{1:i-1})\big)$, where f embeds the context.
- f = **DAN**: ignores order (but the previous word matters far more than one 15 words back). f = **FFNN over concatenated embeddings** (Bengio et al. 2003): parameters grow with context length, and position-specific weights mean "great" in slot 4 shares nothing with "great" in slot 3.
- **RNN:** a cell updates state $\mathbf{h}_t = \tanh(W\mathbf{x}_t + V\mathbf{h}_{t-1} + \mathbf{b})$, sharing parameters across positions and handling arbitrary length.
- **RNN shortcomings:** (1) *vanishing gradients*: saturated tanh, and repeated multiplication by V makes gradients shrink or explode; even LSTMs struggle over 1000-token spans. (2) *Slow*: $O(n)$ sequential steps that can't be parallelized. (3) The state is dominated by recent tokens (Fall 2023 Q1(5)). Transformers fix (1) and (2) with self-attention.

Canvas Exercise 4.0

Q1: a DAN predicting "baseball players play ___" (after "Basketball players play basketball. Tennis players play tennis."). Why might it fail?

Answer: It struggles to encode relations/structure between words, and it can't encode positional information. (It *can* see far-back words.)

Q2: an FFNN over the last 4 words for "Alice bought a pair of sneakers. … she realized they didn't ___".

Answer: It can't look at critical words far back in the sentence.

Traps

- The context count is $c(\text{ctx})$ = how many times ctx appeared *as a context* (followed by something). It isn't necessarily the raw unigram count if the word can end a sentence without a stop symbol.

- One zero factor makes the whole sentence probability 0 under MLE.
- Map OOV words to UNK *before* counting and before scoring.
- Perplexity: divide by the number of *predicted* tokens (include $\langle/s\rangle$ if it's predicted; don't count $\langle s\rangle$).
- Lower perplexity is better; higher log-likelihood is better.

12.6 Practice

Practice

1. Using the Sam corpus, compute $P(\langle s\rangle \text{ Sam I am } \langle/s\rangle)$ under the bigram MLE.
2. A model assigns probabilities 0.5, 0.25, 0.125, 0.5 to a 4-token held-out text. Compute the perplexity.
3. Absolute discounting with $k = 0.5$: the context "I like" was seen 10 times, followed by 4 distinct words, one of them ("pizza") 5 times. Give the discounted term for pizza and λ.
4. Why is the perplexity of a uniform distribution over $|V|$ words equal to $|V|$?

Answers

1. $\frac{1}{3} \cdot P(\text{I}|\text{Sam}) \cdot \frac{2}{3} \cdot \frac{1}{2}$. $P(\text{I}|\text{Sam}) = c(\text{Sam I})/c(\text{Sam}) = 1/2$, so $\frac{1}{3} \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{18}$.
2. Product $= 0.5 \cdot 0.25 \cdot 0.125 \cdot 0.5 = 0.0078125 = 2^{-7}$, so PPL $= 2^{7/4} \approx 3.364$.
3. $(5 - 0.5)/10 = 0.45$; $\lambda = 0.5 \cdot 4/10 = 0.2$.
4. Each $p = 1/|V|$, so the geometric mean of $1/p$ is $|V|$.

TOPIC 13

Self-Attention: Definition and Mechanics

Sources: Lectures 4.2.0 Attention, 4.2.1 Self-Attention, 4.2.2 Multi-Head Self-Attention, 4.2.3 Position Encodings; slides seg-lm6, seg-lm7, seg-lm8, seg-lm9; Exercises 4.1, 4.2; A3 Part 1; Fall 2023 Q1(13).

In plain words

Attention is a soft lookup. A *query* says what you're looking for, each position offers a *key* saying what it is, and you score every key against the query (dot product). A softmax turns the scores into weights, and you return a weighted average of the positions' *values*. **Self-attention** does this for every position at once: each token is simultaneously a query, a key and a value. The output is a new vector per token that mixes in information from the tokens it attended to, a *contextualized* representation. Everything is parallel, and any token can reach any other in one step.

13.1 Attention with one query (lecture 4.2.0)

Three steps

1. **Scores:** $s_i = \mathbf{k}_i \cdot \mathbf{q}$ for each key i.
2. **Softmax:** $\alpha_i = \exp(s_i) / \sum_j \exp(s_j)$.
3. **Output:** $\mathbf{o} = \sum_i \alpha_i \, \mathbf{e}_i$ (weighted sum of the inputs, later of the *values*).

Worked example: lecture: does the sequence contain a `b`? ($e = 3$)

Embeddings $a = [1,0]$, $b = [0,1]$. Sequence a a b a, so keys = embeddings. Query $\mathbf{q} = [0,1]$ ("look for b").

1. Scores: $0, 0, 1, 0$.
2. Softmax with $e = 3$: $(1,1,3,1)/6 = (\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6})$.
3. Output: $\frac{1}{6}[1,0] \cdot 3 + \frac{1}{2}[0,1] = [\frac{1}{2}, \frac{1}{2}]$. A plain average would give $[\frac{3}{4}, \frac{1}{4}]$, so attention amplified the "b-ness".

Sharper attention: multiply keys by $W^K = 10I$. Scores become $0, 0, 10, 0$, so weights are $\left[\frac{1}{e^{10}+3}, \frac{1}{e^{10}+3}, \frac{e^{10}}{e^{10}+3}, \frac{1}{e^{10}+3}\right]$ (**Exercise 4.1** answer), $\approx (0, 0, 1, 0)$.

13.2 Self-attention (lecture 4.2.1)

Scaled dot-product self-attention (Vaswani et al. 2017), one head

Input $X \in \mathbb{R}^{n \times d}$ (one row per token). Parameters $W^Q, W^K \in \mathbb{R}^{d \times d_k}$, $W^V \in \mathbb{R}^{d \times d_v}$.

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \qquad \text{Attention}(Q,K,V) = \underbrace{\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{A \in \mathbb{R}^{n \times n}} V \in \mathbb{R}^{n \times d_v}.$$

- $S = QK^\top$: $S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$, the compatibility of query i with key j.

- Softmax is **row-wise**: row i of A is a probability distribution over the n positions that token i attends to. That's n distributions (Exercise 4.2 Q1).
- Output row $i = \sum_j A_{ij}\mathbf{v}_j$, a weighted combination of the rows of V.
- $\sqrt{d_k}$ scaling keeps dot products from growing with dimension, which would make the softmax overly peaked (lecture: "not too important").
- "API": a sequence of n vectors in, a sequence of n contextualized vectors out.

Worked example: lecture's matrix version

E = rows [1,0],[1,0],[0,1],[1,0] (a a b a). $W^Q = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$, so every row of $Q = EW^Q$ is [0,1] (every token looks for b). $W^K = 10I$, so $K = 10E$. $S = QK^\top$ has every row $= [0, 0, 10, 0]$. Row-wise softmax puts ≈ 1 on position 3 for every token. Output $= A\,EW^V$. Many different parameterizations give the same behaviour; learned weights won't look this clean.

Worked example: full numeric self-attention (verified)

$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $W^Q = I$, $W^K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $W^V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$, $d_k = 2$.

1. $Q = X$; $K = XW^K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}$; $V = XW^V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 2 \end{bmatrix}$.
2. $S = QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 3 \end{bmatrix}$; scaled $S/\sqrt{2} = \begin{bmatrix} 0.707 & 0 & 0.707 \\ 0.707 & 0.707 & 1.414 \\ 1.414 & 0.707 & 2.121 \end{bmatrix}$.
3. Row-wise softmax: $A = \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.248 & 0.248 & 0.504 \\ 0.284 & 0.140 & 0.576 \end{bmatrix}$ (each row sums to 1).
4. Output $AV = \begin{bmatrix} 0.802 & 1.198 \\ 0.752 & 1.504 \\ 0.860 & 1.432 \end{bmatrix}$.

Without the $\sqrt{d_k}$ scaling, row 1 of A would be $(0.422, 0.155, 0.422)$, which is peakier.

Shapes and cost

$X : n \times d$; $Q, K : n \times d_k$; $V : n \times d_v$; $S, A : \mathbf{n} \times \mathbf{n}$; output $n \times d_v$. Batched: prepend the batch dimension, e.g. A is [batch, n, n]. Cost (Exercise 4.2 Q2): computing Q, K, V is $O(nd\,d_k)$; $QK^\top$ is $O(n^2 d_k)$. Total $O(nk_1k_2 + n^2k_2)$ for input size k_1, key size k_2. **Quadratic in sequence length** (memory too) is the main drawback. **Pros vs. RNNs:** no sequential operations (fully parallel), and path length 1 between any two tokens.

13.3 Multi-head self-attention (lecture 4.2.2)

MHSA

One head's softmax tends to be **peaky**, so it can't easily blend two things ("I visited New __" needs both *visited* and *New*). Run h independent heads, each with its own W_i^Q, W_i^K, W_i^V (randomly initialized, so they learn different patterns):

$$\text{head}_i = \text{softmax}\left(\frac{XW_i^Q(XW_i^K)^\top}{\sqrt{d_k}}\right)XW_i^V, \qquad \text{MHSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O.$$

$W^O \in \mathbb{R}^{hd_v \times d}$ projects back to the model dimension, so input and output are both $n \times d$. Illustrated Transformer example: 8 heads with $d_v = 3$, concat gives 24 dims, W^O brings it back down. Standard choice: $d_k = d_v = d/h$. Parameters per MHSA layer (no biases): $3d \cdot (hd_k) + hd_v \cdot d = 4d^2$ when $hd_k = d$.

13.4 Positional encodings (lecture 4.2.3)

Self-attention is **permutation-equivariant**: without position information, a query can't tell where tokens are ("visited" and "ate" look the same from the blank's perspective).

scheme	how
Absolute (learned)	A second embedding table indexed by position $(1, 2, 3, \ldots)$; **add** it to the word embedding at the input. Simple and standard (GPT-3; A3). Drawback: no embedding exists for positions beyond the maximum trained length, and nothing is shared between positions.
Sinusoidal (Vaswani)	Fixed (not learned) sines/cosines at different frequencies, added to the input. Nearby positions get similar vectors (only high-frequency dims differ); distant ones differ more.
Relative (T5)	Inject a learned function of the *distance* $i - j$ directly into the $QK^\top$ scores.
ALiBi	Add $-m \cdot (\text{distance})$ to the scores: a fixed linear penalty with a different slope m per head (some heads local, some global). Very few parameters.
NoPE (2023)	No positional encoding at all. Works for *causal* (past-only) attention, which can count tokens implicitly. Fails for bidirectional attention.

Midterm Fall 2023 Q1(13) (A3 language model)

Possible shapes for the attention map?

Official answer: II. [seq len, seq len]; III. [batch size, seq len, seq len]. Not [batch, seq] (that's one score per token), not [batch, seq, d_{model}] (that's the output), not $d_k \times d_k$ or $d_{model} \times d_{model}$ (those are parameter shapes).

Canvas Exercise 4.2

Q1: number of probability distributions for a length-n sentence:

Answer: n (n^2 values in total). Q2: complexity of the self-attention matrix:

Answer: $O(nk_1k_2 + n^2k_2)$.

Traps

- Softmax over **rows** (over keys j for a fixed query i), not columns.
- A is $n \times n$, independent of d. The output has the value dimension.
- Attention weights are *activations*, not parameters. The parameters are W^Q, W^K, W^V (and W^O).
- Positional encodings are *added* to the embeddings (same dimension), not concatenated, in the lecture/A3 scheme.
- Self-attention alone isn't a Transformer: you also need the FFN, residuals and layer norm (Topic 14).
- If a question gives $e = 3$, use it in the softmax.

13.5 Practice

Practice

1. Keys $[1,0], [0,1], [1,1]$, values = keys, query $[2,0]$, no scaling, e real. Compute the attention weights and output.
2. $n = 10$ tokens, $d = 64$, 4 heads, $d_k = d_v = 16$. Give the shapes of each head's Q, A, head output, the concatenation, and W^O.
3. Apply a causal mask to the full numeric example (each token sees only itself and earlier tokens). Give row 2 of A and row 2 of the output.

Answers

1. Scores $2, 0, 2$: weights $(e^2, 1, e^2)/(2e^2+1) = (0.4683, 0.0634, 0.4683)$. Output $= 0.4683[1,0] + 0.0634[0,1] + 0.4683[1,1] = [0.9366, 0.5317]$.
2. $Q : 10 \times 16$, $A : 10 \times 10$, head out 10×16, concat 10×64, $W^O : 64 \times 64$.
3. Unscaled scores row $2 = (1, 1, -\infty)$, so $A_2 = (0.5, 0.5, 0)$ and the output is $0.5[1,0] + 0.5[0,2] = [0.5, 1]$.

MODULE 5

Transformers and Decoding

TOPIC 14

Transformers: Architecture Details

Sources: Lectures 5.0.0 Transformer Architecture, 5.0.1 Using Transformers, 5.0.3 Transformer Extensions; slides seg-trans1, seg-trans2, seg-trans4; A3 Part 1; Fall 2023 Q1(9–11).

In plain words

A Transformer layer = **multi-head self-attention** (tokens exchange information) + a **position-wise feed-forward network** (each token is processed on its own), each wrapped with a **residual connection** (add the input back) and **layer normalization** (rescale). Every layer maps n vectors of size d_{model} to n vectors of size d_{model}, so layers stack. Attention gets the headlines, but most parameters and compute live in the feed-forward networks.

14.1 One Transformer layer (encoder block)

Block structure (Vaswani et al. 2017, "post-norm" as drawn in lecture)

$$\mathbf{H} = \text{LayerNorm}\big(X + \text{MHSA}(X)\big)$$
$$\text{Out} = \text{LayerNorm}\big(\mathbf{H} + \text{FFN}(\mathbf{H})\big), \qquad \text{FFN}(\mathbf{h}) = W_2\,\text{ReLU}(W_1\mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2$$

- **FFN is applied to each position independently** (same weights for every token). It's a plain one-hidden-layer network $d_{model} \to d_{ff} \to d_{model}$ (lecture calls it $d_{internal}$).
- **Residual connections**: add the sublayer's input to its output. Gradients flow easily and the model can effectively "skip" layers it doesn't need. This forces sublayer outputs to have dimension d_{model}.
- **Layer normalization**: rescale each token's vector (mean 0, variance 1, then learned scale/shift). It keeps layers on the same scale so one learning rate works for deep stacks.
- **Input:** token embeddings + positional encodings (Topic 13). **Output:** contextualized vectors, one per token.

Dimensions

quantity	symbol	Vaswani base / notes
model (residual stream) width	d_{model}	512 (input = output of each block)
query/key size per head	d_k	64 (must match between Q and K)
value size per head	d_v	64 (concat hd_v, then W^O back to d_{model})
heads	h	8 ($hd_k = d_{model}$)
FFN hidden	d_{ff}	2048 ($4 \times d_{model}$)
layers	N	6

GPT-3 scaling: d_{model} grows to about 12,288, but d_{head} stays at 128. In OPT-175B, MHSA is about 17% of the FLOPs and the **FFN about 80%**. The huge FFNs are thought to store much of the model's factual knowledge.

14.2 Parameter counting

Per layer (biases and LayerNorm in parentheses)

- MHSA: W^Q, W^K, W^V: $3 \cdot d_{model} \cdot hd_k$; W^O: $hd_v \cdot d_{model}$. With $hd_k = hd_v = d_{model}$: $\mathbf{4d^2_{model}}$ $(+4d_{model})$.
- FFN: $\mathbf{2\, d_{model}\, d_{ff}}$ $(+d_{ff} + d_{model})$.
- 2 LayerNorms: $(4d_{model})$.
- Plus embeddings $|V|d_{model}$, learned positions $n_{max}d_{model}$, output layer $d_{model}|V|$ (often tied to the embeddings).

Vaswani base: $4(512^2) = 1{,}048{,}576$ for attention, $2(512)(2048) = 2{,}097{,}152$ for the FFN. The FFN is twice the attention, and that's 3,145,728 per layer (3,152,384 with biases + LN).

Midterm Fall 2023 Q1(9)–(11)

(9) A "3-layer Transformer" whose middle layer has only Multi-Head Attention + Add&Norm (its FFN removed). Parameters vs. a standard 3-layer?

Official answer: C. Fewer (one FFN missing). (10) Performance across many tasks?

Official answer: C. Worse.

(11) A 2-layer Transformer where each layer is MHA + FFN + FFN (two FFNs) vs. a standard **4-layer**:

Official answer: C. Fewer. 2 MHA + 4 FFN vs. 4 MHA + 4 FFN: same number of FFNs, but two fewer attention blocks.

14.3 Using Transformers (lecture 5.0.1)

- **API:** a sequence of vectors in, a same-length sequence of contextualized vectors out (like RNN states).
- **Per-token classification** (POS tagging, A3 Part 1 letter counting): linear + softmax on each output vector.
- **Sequence classification** (sentiment): either average the outputs then linear + softmax (DAN-style), or (standard) prepend a **[CLS]** token and classify from its output vector. [CLS] attends to every token, so it learns a task-specific weighting. That's smarter than average pooling.
- **Sentence-pair classification** (entailment): feed both sentences in one sequence. Self-attention aligns words across the two (*driving* vs. *walking*). This was a headline BERT result (Topic 16).
- The original Transformer is **encoder–decoder** (for translation). Language models use a decoder-only stack with a causal mask (Topic 15).

14.4 A3 Part 1: letter counting

Task: for each of 20 characters, predict how many times that character occurred **before** (0, 1, or $\geq$2): 3 classes at every position. Example: "i like movies a lot" $\rightarrow$ `00010010002102021102`. The BEFOREAFTER variant counts both directions.

- Implement from scratch (only Linear, softmax, nonlinearities): **single-head attention** $\rightarrow$ **residual** $\rightarrow$ **Linear–ReLU–Linear** $\rightarrow$ **residual**. LayerNorm is optional for a shallow net. Queries/keys/-

values come from Linear layers; use matmul + transpose; divide by $\sqrt{d_k}$.

- Output: log-probabilities $[20 \times 3]$ + the attention maps; NLLLoss over all positions at once.
- **Positional encodings needed** for BEFORE (without them the model sees a bag of characters and can't tell before from after). They're learned `nn.Embedding` positions added to character embeddings: $\text{emb}_{char}(t) + \text{emb}_{pos}(0)$.
- Targets: $> 95\%$ (reference $> 98\%$ with 1–2 layers). If stuck at about 70%, layers stored in a plain Python list aren't registered (use `nn.ModuleList`).

14.5 Efficiency extensions (lecture 5.0.3, MC fodder)

- **Scaling laws** (Kaplan et al. 2020): test loss falls as a smooth power law (log-linear) in compute, data and parameters. Transformers keep improving with scale, unlike RNNs.
- **Quadratic attention** is the bottleneck. **Performer** multiplies $K^\top V$ first (a linear-time approximation, not the same computation). **Longformer** uses sparse attention patterns (sliding window, dilated, global). These mostly save *memory*, not time, and approximations usually underperform full attention.
- Much recent progress is systems work (GPU memory handling) for longer contexts.

Traps

- "Transformer" $\neq$ "self-attention": a block also has the FFN, residuals and layer norm.
- The FFN is position-wise: it does *not* mix information across tokens. Only attention does.
- Residuals require matching dimensions, so the MHSA output (after W^O) and the FFN output must be d_{model}.
- d_k can be smaller than d_{model}; W^Q/W^K needn't be square.
- Removing a sublayer removes parameters. Count MHSA blocks and FFN blocks separately (Fall 2023 Q9–11).

14.6 Practice

Practice

1. $d_{model} = 256$, $h = 4$, $d_k = d_v = 64$, $d_{ff} = 1024$, 6 layers, vocabulary 10,000, untied output, no biases/LN/positions. Total parameters?
2. Which costs more FLOPs per token in a large Transformer: attention projections or the FFN? Why?
3. A model replaces each FFN with a second MHSA block. More or fewer parameters with the standard $d_{ff} = 4d_{model}$?

Answers

1. Per layer $4(256^2) + 2(256)(1024) = 262{,}144 + 524{,}288 = 786{,}432$; $\times 6 = 4{,}718{,}592$; embeddings + output $2 \times 10{,}000 \times 256 = 5{,}120{,}000$; total 9,838,592.
2. The FFN ($2d\,d_{ff} = 8d^2$ per token vs. $4d^2$ for the projections), plus attention's n^2 term for long sequences.
3. Fewer: an MHSA block is $4d^2$ vs. $8d^2$ for the FFN.

TOPIC 15

Transformer Language Modeling (Assignment 3)

Sources: Lectures 5.0.2 Transformer Language Modeling, 4.0.2 LM Evaluation; slides seg-trans3, seg-lm3; Assignment 3 Part 2; Fall 2023 Q1(13–14).

In plain words

Feed the text into a Transformer and, at **every** position, predict the *next* token from that position's output vector (linear layer + softmax over the vocabulary). Training uses all positions at once: inputs are the sequence, targets are the same sequence shifted left by one, and the loss is the summed (or averaged) negative log-probability of the true next tokens. A **causal mask** stops a position from attending to future tokens. Otherwise the model just copies the answer. Evaluate with perplexity on held-out text.

15.1 Model

Next-token distribution

With $\mathbf{h}_i$ the final-layer vector at position i:

$$P(w_{i+1} = w \mid w_{1:i}) = \frac{\exp(\mathbf{w}_w \cdot \mathbf{h}_i)}{\sum_{w' \in V} \exp(\mathbf{w}_{w'} \cdot \mathbf{h}_i)}$$

equivalently $P(\cdot \mid w_{1:i}) = \text{softmax}(W\mathbf{h}_i)$ with $W \in \mathbb{R}^{|V| \times d_{model}}$. Rows of W act like (output) word embeddings: the model puts mass on words compatible with $\mathbf{h}_i$.

15.2 Training

Shifted targets, simultaneous prediction

1. Sequence "I saw the dog". Input = $\langle s \rangle$ I saw the; targets = I saw the dog. (A3: **space is the start symbol**. Feed space + first 19 chars and predict all 20.)
2. Run the Transformer **with a causal mask**. The output is $[n, |V|]$ log-probabilities (batched: $[B, n, |V|]$).
3. Loss $= \sum_i -\log P(\text{target}_i \mid \text{prefix})$ = NLL, i.e. the log-likelihood of the whole chunk (log of a product = sum of logs). `nn.NLLLoss(log_probs [n,|V|], targets [n])`. NLLLoss averages by default.
4. Batched: reshape $[B, n, |V|] \to [Bn, |V|]$ and targets $[B, n] \to [Bn]$ before NLLLoss.
5. zero_grad → forward → loss → backward → step (Adam).

Batching (lecture): split the stream into chunks (e.g. length 5 or 20) and stack independent chunks along a batch dimension. That is *not* the same as training on one long sequence: each chunk sees only its own tokens, so positions and attention never span chunks.

15.3 The causal mask

Why and how

Why: without it, position i attends to $i+1$, copies the next token into its representation, and predicts it perfectly. Perplexity drops near 1 and the A3 sanity check fails. That isn't language modeling: the model must represent $P(w_i \mid w_1, \ldots, w_{i-1})$.
How: before the softmax, add M with $M_{ij} = -\infty$ for $j > i$ (future) and 0 otherwise. Then $e^{-\infty} = 0$, so future tokens get zero attention.

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

PyTorch: `torch.triu(torch.ones(n,n), diagonal=1)` marks the future (True ⇒ `masked_fill(-inf)`), or pass a triangular $0/-\infty$ `mask` to `nn.TransformerEncoder`. Row 1 of A is always $[1, 0, 0, \ldots]$: the first token attends only to itself.

15.4 Inference and scoring (A3 `NeuralLanguageModel`)

- `get_next_char_log_probs(context)`: input = space + context (truncate to the last n_{max} positions). Return the **last row** of log-probabilities, a full length-27 vector of natural logs. **Not** an argmax: evaluation needs the probabilities.
- `get_log_prob_sequence(next_chars, context)` = $\sum_k \log P(c_k \mid \text{context}, c_{1:k-1})$: score each character *before* appending it to the context.
- Call `model.eval()` (turns off dropout) and use `torch.no_grad()`.
- **Normalization check:** each returned distribution must sum to 1. Range check: probabilities in $[0, 1]$.

15.5 Evaluation: perplexity on A3

A3 numbers

Dev set 500 characters, 27 types. **Uniform model:** total log-prob $500 \ln(1/27) = -1644$, average -3.296, perplexity $= e^{3.296} = 27$. Exponentiating the average log-probability gives $1/27$, the *inverse* of perplexity. Requirement: PPL ≤ 7 in under 10 minutes; reference 6.3. **Your run** (4 layers, $d_{model} = 32$, $d_{internal} = 32$, chunks of 20, Adam 10^{-3}, 10 epochs): log-prob -864.02, average -1.728, **PPL** $= e^{1.728} = 5.63$.

$$\text{PPL} = \exp\left(-\tfrac{1}{N}\textstyle\sum_i \ln P(c_i \mid c_{<i})\right).$$

15.6 Your A3 model, piece by piece

```
x = embedding(indices)              # [T] -> [T, d_model]   (27 x 32 table)
x = positional_embedding(x)         # + learned position vectors (20 x 32)
for block in blocks:                # 4 x TransformerBlock
```

```
    a = causal_attention(x)        # Q,K: Linear(32,32); V: Linear(32,32); /sqrt(d_k); triu mask
    x = LayerNorm(x + a)           # residual + norm
    x = LayerNorm(x + FF2(ReLU(FF1(x))))
logits = output_layer(x)           # [T, 27]
log_probs = log_softmax(logits, dim=1)    # NLLLoss with targets [T]
```

Single-head attention without W^O (since V already maps to d_{model}). Total parameters with biases: 24,027 (5,408 per block).

15.7 Past exam problems

Midterm Fall 2023 Q1(13)

Possible attention-map shapes when training an A3 LM:

Official answer: II. [seq len, seq len]; III. [batch size, seq len, seq len].

Midterm Fall 2023 Q1(14)

Possible shape for the model outputs fed to the loss:

Official answer: V. [batch size, seq len, vocab size]. Not [batch, vocab] (the model predicts at *every* position), not [batch, seq, d_{model}] (hidden states before the output layer). Strictly, NLLLoss wants it flattened to $[B \cdot n, |V|]$, but V is the model's output shape.

Worked example: causal attention on 3 tokens

Unscaled scores $S = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 3 \end{bmatrix}$ (from Topic 13). Masked: $\begin{bmatrix} 1 & -\infty & -\infty \\ 1 & 1 & -\infty \\ 2 & 1 & 3 \end{bmatrix}$. Row-softmax: $A = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.245 & 0.090 & 0.665 \end{bmatrix}$. Token 3 (the last) is unaffected, since it can already see everything.

Traps

- **Off-by-one:** the output at position i predicts token $i+1$. Inputs start with the start symbol, and targets are the inputs shifted by one.
- The causal mask uses $-\infty$ *before* softmax (not 0 after). Masking with 0 in score space still gives weight $e^0 = 1$.
- Perplexity uses natural log consistently (A3 returns base-e logs). Average over the number of *predicted* characters.
- Return log-probabilities, not probabilities, when the API says log.
- The LM output is $[\ldots, |V|]$, not $[\ldots, d_{model}]$.

15.8 Practice

Practice

1. A chunk "hello" in A3's setup: write the input and target strings.
2. Your LM assigns the 4 true next characters probabilities 0.5, 0.2, 0.1, 0.4. Total log-probability, average NLL, perplexity?
3. A classmate's model reaches dev perplexity 1.02 after one epoch. Most likely bug?
4. Batch of 16 chunks of length 20, vocabulary 27. Give the shapes of the input indices, the output log-probs, the targets, and what you pass to NLLLoss.

Answers

1. Input "␣hell", target "hello".
2. $\ln 0.5 + \ln 0.2 + \ln 0.1 + \ln 0.4 = -0.6931 - 1.6094 - 2.3026 - 0.9163 = -5.5215$; average NLL 1.3804; PPL $= e^{1.3804} = 3.976$.
3. A missing or incorrect causal mask: the model sees the future.
4. $[16, 20]$; $[16, 20, 27]$; $[16, 20]$; flatten to $[320, 27]$ and $[320]$.

TOPIC 18

Decoding: Greedy, Beam Search, Nucleus Sampling

Sources: Lectures 5.1.0 Beam Search, 5.1.1 Nucleus Sampling; slides seg-infer1, seg-infer2; Exercise 5.0; Fall 2023 Q1(16–18).

In plain words

A trained LM gives a next-token distribution. Turning that into text needs a *decoding strategy*. **Greedy**: always take the top token. **Beam search**: keep the k best partial sequences at each step, which finds higher-probability outputs; good for translation. **Sampling**: draw at random, for variety, but the long tail of junk tokens eventually gets sampled. **Nucleus (top-p) sampling**: sample only from the smallest set of top tokens covering probability p, renormalized. Varied but sane; it's what chat models typically use.

18.1 Exhaustive vs. greedy vs. beam

- Exhaustive search over length-n outputs: $|V|^n$ sequences, exponential.
- **Greedy:** $\hat{w}_t = \arg\max_w P(w \mid \hat{w}_{<t})$. Cheap and works OK, but can get derailed by an early locally-good choice.

Beam search with beam size k

1. Step 1: score all $|V|$ first tokens; keep the top k.
2. Each step: extend every beam hypothesis by every token, giving $k|V|$ candidates scored by **total sequence probability** (product, or sum of logs).
3. Keep the top k candidates overall (not top k per parent). Repeat until length n / end-of-sequence.
4. Return the best (or all k, if asked).

Cost: about kn Transformer calls (one per beam item per step) and $k|V|n$ hypotheses considered, vs. $|V|^n$ for exhaustive. $k = 1$ is greedy.

When to use: conditional generation with one right answer (machine translation). Less important for very large LMs, and expensive. **Degeneration:** beam search on open-ended text falls into repetition loops ("Universidad Nacional Autónoma de México/…" repeated). Each step is locally likely, and the loop has high overall probability.

Worked example: beam beats greedy

$P(w_1)$: A 0.5, B 0.4, C 0.1. $P(\cdot|A)$: x 0.4, y 0.3, z 0.3. $P(\cdot|B)$: x 0.9, y 0.05, z 0.05. Greedy: A then x, giving $0.5 \times 0.4 = 0.20$. Beam $k = 2$: keep {A, B}. Candidates: Ax 0.20, Ay 0.15, Az 0.15, Bx 0.36, By 0.02, Bz 0.02. Keep {Bx 0.36, Ax 0.20}, so the best is **B x** (0.36), which beats greedy.

18.2 Sampling and nucleus sampling

Nucleus (top-p) sampling (Holtzman et al. 2020)

1. Sort tokens by probability, high to low.
2. Keep the smallest prefix whose cumulative probability is $\geq p$ (the "nucleus").
3. **Renormalize** within the nucleus (divide by the kept mass) and sample.

Why: if the tail holds 10% of the mass, pure sampling picks a tail word about every 10 tokens and goes off the rails. Results: nucleus sampling has moderate perplexity (not as low as greedy/beam) but far less repetition (low self-BLEU), and good human-judged quality.

Canvas Exercise 5.0

Q1 true of nucleus sampling:

Answer: the most likely word *may* be chosen. Not "any word may be chosen" (the tail is cut), not "the most likely word *must* be chosen". Not "always introduces randomness at every step": if the top token alone has probability $\geq p$, it is chosen deterministically.
Q2: the: 0.7, a: 0.1, computer: 0.1, tablet: 0.08, phone: 0.01, server: 0.01, with $p = 0.95$. Cumulative: 0.7, 0.8, 0.9, **0.98** ≥ 0.95, so keep the first 4. $P(\text{tablet}) =$

Answer: $0.08/0.98 \approx 0.0816$.

18.3 Past exam: Fall 2023 Q1(16–18)

Midterm Fall 2023 Q1(16)–(18)

$P(a) = 0.8$, $P(the) = 0.15$, $P(dog) = 0.05$. $P(\cdot|a)$: a .01, the .01, dog .98. $P(\cdot|the)$: a .15, the .01, dog .84. $P(\cdot|dog)$: a .3, the .5, dog .2. Exactly two tokens are generated.
(16) Beam size 2. Step 1 keeps {a .8, the .15}. Step 2 candidates: a a .008, a the .008, **a dog .784**, the a .0225, the the .0015, **the dog .126**.

Official answer: a dog, the dog.
(17) How many sequences could pure sampling return? All 9 have nonzero probability:

Official answer: 9.
(18) Nucleus $p = 0.9$: step 1: $0.8 < 0.9$, add "the" to reach 0.95, so {a, the}. After a: dog $0.98 \geq 0.9$, so {dog}. After the: dog 0.84, add a to reach 0.99, so {dog, a}. Sequences: a dog; the dog; the a, so

Official answer: 3.

Traps

- Beam keeps the top k *overall* among all extensions, ranked by the product of probabilities so far, not just the last token's probability.
- Nucleus: include the token that *crosses* p ($\geq p$), then renormalize by the kept mass (0.98, not p=0.95).
- Nucleus is decided *per step*; the nucleus set changes with context.
- Count possible sequences path by path: each first token contributes the size of its own next-step nucleus (Q18: $1 + 2 = 3$), not 2×2.
- Greedy = beam with $k = 1$; beam with $k = |V|^n$ (unbounded) = exhaustive.

18.4 Practice

Practice

1. In the Fall 2023 setup, what does greedy decoding return, and with what probability?
2. Same setup, nucleus $p = 0.8$. How many sequences are possible?
3. Distribution: x .5, y .3, z .15, w .05, $p = 0.75$. What is the nucleus and the renormalized probability of y?
4. Beam size 3, $|V| = 50{,}000$, length 20. How many Transformer calls and hypotheses are scored (approximately)?

Answers

1. a, then dog: 0.784. 2. Step 1: a ($0.8 \geq 0.8$), so {a}. After a: {dog}. So 1 sequence. 3. Cumulative 0.5, $0.8 \geq 0.75$, so {x, y}; $P(y) = 0.3/0.8 = 0.375$. 4. About $kn = 60$ calls; $k|V|n = 3 \times 10^6$ hypotheses.

MODULE 6

Pre-training and Tokenization

TOPIC 16

Encoders: Masked Language Modeling and BERT

Sources: Lectures 6.0.0 BERT: Masked Language Modeling, 6.0.1 BERT: Model and Applications, 6.1.0 Seq2seq Models, 6.1.1 BART, 6.1.2 T5; slides seg-68, seg-69, seg-s2s1–3; Exercises 6.0, 6.1; Fall 2023 Q1(12).

In plain words

To understand a word you want context from *both* sides, but a normal (left-to-right) LM can't look right, or it would see the answer. **Masked language modeling** hides about 15% of the tokens (replaces them with [MASK]) and trains a *bidirectional* Transformer encoder to fill them in from everything else. Nothing to cheat from, since the answer isn't in the input. Pre-train BERT this way on huge text, then **fine-tune** the whole network (with a small classifier on the [CLS] vector) on your task. BERT is great at *analysis* (classification, tagging, sentence pairs) but can't naturally generate text.

16.1 Why not a Transformer LM? Why masking?

- **ELMo** (2018): forward and backward LSTM LMs, *concatenated*. Two independent one-sided views, not deeply merged.
- **GPT**: a one-sided (causal) Transformer LM. It only looks back.
- A bidirectional Transformer trained to predict the next word would **cheat**: position i attends to word $i+1$.
- **MLM fix:** "John visited [MASK] yesterday" $\rightarrow$ predict *Madagascar*. The model sees all other words in both directions but not the target. Only the masked positions incur loss (about 15%). If you masked everything there'd be nothing to predict from.

16.2 BERT pre-training

BERT objective = MLM + next sentence prediction (NSP)

Input: `[CLS] text chunk 1 [SEP] text chunk 2`. Embeddings = word piece + **position** (learned integer embedding) + **segment** (chunk 1 vs. 2).

1. **MLM:** mask 15% of the tokens and predict them (softmax over the vocabulary at those positions). In the BERT paper, the chosen tokens are replaced with [MASK] 80% of the time, a random token 10%, and left unchanged 10%. (The lecture just says "mask out 15%"; the 80/10/10 detail is from Devlin et al. and explains Fall 2023 Q12 option C.)
2. **NSP:** 50% of the time chunk 2 truly follows chunk 1; 50% it's a random chunk ("NotNext"). Predict IsNext/NotNext from the **[CLS]** output vector. Later work (RoBERTa) found NSP isn't important.

Model sizes

Both are Transformer encoders (bidirectional self-attention, no causal mask). **BERT-Base:** 12 layers, $d_{model} = 768$, 12 heads, about 110M parameters. **BERT-Large:** 24 layers, $d_{model} = 1024$, 16 heads, **340M** parameters. Input tokens are **WordPieces** (Topic 17).

16.3 Applying BERT: fine-tuning

Fine-tuning recipe

1. Feed the input (single sentence, or sentence pair joined with [SEP]).
2. **Classification:** take the final [CLS] vector → linear/small FFN → softmax. **Tagging:** classify each token's output vector (e.g. NER tags).
3. Train the new head **and all of BERT** on task data for a few epochs with a **small learning rate** (plus tricks such as warmup). Top layers change most, lower layers little, which preserves the pre-trained knowledge.

- **Fine-tuning helps BERT** (unlike ELMo, where fine-tuning vs. frozen features hurt). BERT is rarely used as frozen embeddings.
- **Sentence-pair tasks** (paraphrase, entailment) improved most. Self-attention aligns the two sentences. Most of this is learned during fine-tuning, not from NSP.
- **GLUE benchmark:** BERT-Large beat task-specific state-of-the-art models and ELMo/GPT by large margins.
- **Analysis** (Clark et al.): some heads attend to the previous/next token; some attend broadly to [SEP]/periods; some track syntax (verbs → direct objects, nouns → modifiers), learned without supervision.
- **RoBERTa:** about 10× more data, **dynamic masking** (different masks each epoch), drops NSP. Also **whole-word masking** (mask all pieces of a word).
- **Can't generate well:** you'd append [MASK] and refill repeatedly, which is $O(n^2)$ over the output length, and parallel fills may not be coherent. BERT is meant for **analysis** tasks.

16.4 Encoder vs. decoder vs. encoder–decoder

	attention	pre-training	best for
Encoder (BERT, RoBERTa)	bidirectional	MLM (+NSP)	classification, tagging, sentence pairs
Decoder (GPT, A3 LM)	causal (past only)	next-token LM	generation
Enc–dec (Transformer, BART, T5)	encoder bidirectional; decoder causal + **cross-attention** to the encoder	denoising (BART: reconstruct full text; T5: predict only masked spans)	seq2seq: translation, summarization, QA

- **Seq2seq:** $P(\mathbf{y}|\mathbf{x}) = \prod_i P(y_i \mid \mathbf{x}, y_{1:i-1})$. The encoder runs **once**; the decoder generates left to right until a STOP token, feeding back each output. Two models with separate parameters (and possibly vocabularies). Lower-entropy than language modeling (MT is "difficult, not impossible").

- **BART** (Lewis et al., Meta): corrupt the input (mask spans, delete, permute, . . .), encode it, and the decoder outputs the **entire clean text**. A superset of BERT (the encoder alone can be used like BERT). Strong at **summarization**.
- **T5** (Google): mask spans, predict **only the masked spans**. C4 dataset: more *unique* data helps (repeating less data is worse). Multitask/unified text-to-text, e.g. UnifiedQA.

16.5 Past exam / exercises

Midterm Fall 2023 Q1(12)

What would most likely improve fine-tuned accuracy of a pre-trained BERT? A. Masking more than 15% of tokens; B. only left-to-right encoding; C. replacing 100% of masked tokens with random words.

Official answer: A. B removes bidirectionality (BERT's key advantage). C makes inputs noisy and inconsistent: the model can never trust any token and never sees [MASK]. (A is the official answer; later work found somewhat higher masking rates can help.)

Canvas Exercise 6.0

Q1 BERT:

Answer: masks 15% of tokens and predicts them; includes an NSP classification loss. It is *not* trained to generate 100% of tokens, there's no attention-penalty loss, and no grammaticality/noising loss (that's BART). Q2 RoBERTa:

Answer: trains on more data; changes the masks every epoch.

Canvas Exercise 6.1 (seq2seq)

Q1 true:

Answer: runtime quadratic in output length; linear in decoder vocabulary size if layers/lengths are fixed; any classifier network can model $P(y_i|\mathbf{x}, y_{<i})$. Not "decode out of order". Q2 why Transformers:

Answer: reuse computation across decoding steps; pass information over long distances better than RNNs; the decoder can use the whole output prefix. Multiple decoder steps *can't* run in parallel at inference.

Traps

- MLM loss is only on masked positions. Next-token LM loss is on every position.
- BERT is bidirectional (no causal mask); GPT/A3 LM is causal.
- NSP uses [CLS]; MLM uses the masked positions' outputs.
- "BERT is a language model" in the GPT sense is false. It's a *masked* LM and doesn't define $P(\text{sentence})$ left to right.
- BART reconstructs the full sequence; T5 outputs only the masked spans.

16.6 Practice

Practice

1. A 200-token BERT input with 15% masking: about how many positions contribute MLM loss? With a GPT-style LM?
2. Why can't you train BERT's bidirectional encoder with a next-token objective?
3. Which model for: (a) sentiment classification, (b) news summarization, (c) story continuation?

Answers

1. About 30; GPT: 200 (every position predicts its next token). 2. With bidirectional attention, each position can attend to the next token and copy it, a trivial solution. 3. (a) BERT/encoder with [CLS] head; (b) BART/T5 (enc–dec); (c) decoder LM (GPT) with nucleus sampling.

TOPIC 17

Subword Tokenization (BPE, WordPiece)

Sources: Lecture 6.2.0 Word Piece and Byte Pair Encoding; 3.1.1 Other Word Embedding Methods (fastText/subwords); slides seg-subword-segmentation; Exercise 6.2; Fall 2023 Q2(c–e), Q1(15).

In plain words

Word vocabularies are either too small (lots of UNKs, copying needed) or too big (slow softmax). Characters are too fine (sequences become very long). **Subwords** are the compromise: frequent words stay whole, and rare words split into meaningful pieces (*re_ fueling*, *un_ princi_ pled*). Any word can be represented, so there are no UNKs, and related words share pieces (*dog*, *dog s*).

17.1 Byte Pair Encoding (BPE; Sennrich et al. 2016)

Learning BPE merges

1. Start with every character (byte) as its own symbol (often with an end-of-word marker _). Word counts come from a large corpus.
2. Count adjacent symbol pairs, weighted by word frequency.
3. **Merge the most frequent pair** into a new symbol; add it to the vocabulary.
4. Repeat for a fixed number of merges. 30k merges gives a vocabulary of about 30,000 pieces, including many whole words.

To segment new text, apply the learned merges in order (or greedy longest match against the vocabulary, as the exam does).

Worked example: BPE on {low:5, lower:2, newest:6, widest:3} (end marker _)

1. Pair counts: (e,s)=9, (s,t)=9, (t,_)=9, (w,e)=8, … Tie: take the first, **es**.
2. (es,t)=9, so merge **est**. 3. (est,_)=9, so merge **est_**.
3. (l,o)=7, so **lo**. 5. (lo,w)=7, so **low**. 6. (n,e)=6 (tie with (e,w), (w,est_)), so **ne**.

Now “newest” = ne w est_, “lowest” (unseen) = low est_. Ties must be broken by a stated rule.

17.2 WordPiece / unigram LM (SentencePiece)

- Choose pieces that best explain the corpus under a language model: merge (or keep) the pieces that most improve LM likelihood/perplexity, until the target vocabulary size is reached. Google’s SentencePiece library implements it (Google Translate). Lecture: “unigram LM”. BERT uses WordPieces.
- **Bostrom & Durrett (2020):** BPE produces less linguistically plausible units. Unigram LM gives morphological splits (*tri cycle s*) and some evidence of better pre-trained models.
- Ensembling across multiple tokenizations is also explored.

17.3 Greedy (longest-match) tokenization, the exam algorithm

Greedy segmentation

At each position, take the **longest** vocabulary piece that matches starting there. Emit it and continue after it. (If nothing matches, fall back to a single character/UNK.) Greedy is *not* guaranteed to use the fewest pieces.

Canvas Exercise 6.2

Q1 advantages:

Answer: any word can be represented; a plural and its singular might differ only in the token "s". Not "sequences get shorter" (they stay the same or get longer); not "misspellings get the same representation" (no guarantee).
Q2 vocabulary {A, AA, AAB, ABB, BAAA}, greedy on AABAAA:

Answer: AAB AA A (no AABA...; then AAA isn't in the vocabulary but AA is; then A).
Q3 fewest pieces:

Answer: AA BAAA (2 pieces vs. greedy's 3).

Midterm Fall 2023 Q2(c)–(e)

Subword vocabulary {good, x, q, not, gre, at, great} with greedy tokenization. Data: `goodx+`, `great+`, `not greatq−`, `not good−`.
(c) Segmentations: goodx → good x; great → great; greatq → great q; not → not. New BoW vocabulary = the 7 subwords in order (good, x, q, not, gre, at, great). Features:

Official answer: (1,1,0,0,0,0,0), (0,0,0,0,0,0,1), (0,0,1,1,0,0,1), (1,0,0,1,0,0,0). (The question meant features for each example; partial credit if computed for the vocabulary.)
(d) `greqat` →

Official answer: gre q at ("great" doesn't match because of the q).
(e) Typos anywhere in a word: subword tokenization vs. edit-distance repair?

Official answer: Typo repair is generally less fragile. Subwords split typo'd words oddly (gre q at loses "great"), while repair maps it back.

Midterm Fall 2023 Q1(15)

Train on singular nouns, test on plurals:

Official answer: stemming, subword tokenization, pre-trained embeddings help. dog/dogs share the piece *dog*.

Traps

- Greedy = longest match from the left, piece by piece. Don't search for the globally fewest pieces unless asked.
- Subword tokenization never makes sequences shorter than word tokenization.
- BPE merges are learned by *frequency*; WordPiece/unigram LM by *likelihood*.
- After segmenting, a BoW vocabulary over subwords counts *pieces*. Use the vocabulary order given.

- The underscore marks a word boundary (start of a word in the lecture's notation, *_eco tax*); continuation pieces have no underscore.

17.4 Practice

Practice

1. Vocabulary {un, u, n, h, a, p, py, happy, ha, ppy}. Greedy-tokenize "unhappy" and "uhappy".
2. Continue the BPE example: what is merge 7 (ties broken by first occurrence in the order low, lower, newest, widest)?
3. Give one reason subwords help a sentiment model on A2's typo dev set, and one reason they might not.

Answers

1. un happy; u happy.
2. After merge 6: low_ (5), low e r _ (2), ne w est_ (6), w i d est_ (3). Pair counts: (low,_)=5, (low,e)=2, (e,r)=2, (r,_)=2, **(ne,w)=6**, **(w,est_)=6**, (w,i)=3, (i,d)=3, (d,est_)=3. Tie at 6; (ne,w) occurs first, so merge 7 is **new** (and merge 8 is newest_).
3. Helps: a typo'd word still shares most pieces with the correct word (A2 typos never touch the first 3 characters). Might not: a typo can change the segmentation entirely, giving pieces with unrelated embeddings.

MODULE 7

POS Tagging, HMMs, Parsing

TOPIC 19

POS Tagging and Its Ambiguities

Sources: Lectures 7.0.0 Part-of-Speech Tagging, 7.1.0 Sequence Labeling, Tagging with Classifiers, 7.3.1 HMMs for POS Tagging; slides seg-22–seg-26; Exercises 7.0, 7.1.

In plain words

POS tagging assigns each word a grammatical category (noun, verb, . . .). Many words can take several tags (*record*, *arms*, *raises*), and the right one depends on context. The number of possible tag sequences grows multiplicatively, so even short sentences have dozens of readings, some grammatical but absurd. Classifiers that tag each word independently can produce incoherent sequences. Sequence models (HMMs, CRFs) score whole sequences. **You won't be asked to know tag definitions**; they'll be provided. You must reason about ambiguity and count/evaluate readings.

19.1 Tags (lecture crash course)

- **Open class** (new words join): nouns (proper: IBM, Italy; common: cat, snow), verbs, adjectives, adverbs (swiftly).
- **Closed class** (function words): determiners (the, some), conjunctions (and, or), pronouns, auxiliaries (*had* gone), modals (could), prepositions (up a mountain), particles (made *up* the story; verb–particle construction).
- Penn Treebank: 45 fine-grained tags, e.g. NNP (proper noun), NNS (plural noun), VBD (past), VBN (participle), VBZ (3rd-singular present), VBP (non-3rd present), VB (base form).

Why tag? Pronunciation (*REcord* vs. *reCORD*), information extraction (arms as noun vs. verb), and a building block for parsing.

19.2 The "Fed raises interest rates 0.5 percent" example

Worked example: counting readings

Fed	raises	interest	rates	0.5	percent
NNP, VBD, VBN	NNS, VBZ	NN, VBP, VB	NNS, VBZ	CD	NN

Number of tag sequences $= 3 \times 2 \times 3 \times 2 \times 1 \times 1 =$ **36**.
Intended: Fed/NNP raises/VBZ interest/NN rates/NNS ("the Fed raises [interest rates]"). The noun–noun compound *interest rates* is fine; *interest* is not an adjective.
Syntactically valid but absurd: Fed/NNP raises/NNS interest/VBP rates/NNS: "[Fed raises] interest [the rates] 0.5 percent" (the raises make the rates interested). Syntax permits it; only world knowledge (rates are inanimate) rules it out. That's what makes tagging and parsing hard for machines.

19.3 Tagging with classifiers (lecture 7.1.0)

- Sequence labeling: input $\mathbf{x} = (x_1, \ldots, x_n)$, output $\mathbf{y} = (y_1, \ldots, y_n)$, same length. This is **structured** classification.
- Independent LR per position: $P(y_i = y \mid \mathbf{x}, i)$. Features must depend on the **position** i. Whole-sentence BoW features are identical for every position, so every word would get the same tag distribution.
- Good features (DF-style indicators): CurrWord=*interest* & tag=NN; PrevWord=*raises*; NextWord=*rates*; prefixes/suffixes (-ing, -ed, -s); capitalization; digits; word length; seen-before?
- **Problem:** independent predictions can be *incoherent* (NNS followed by VBN, etc.). Sequence models fix this: **HMMs** (generative, Topic 20) and CRFs (discriminative; not on the current exam).
- Feedforward tagger (Botha et al.): concatenate embeddings of previous/current/next word (+ character n-grams). Strong across many languages.

Accuracy facts (Penn Treebank)

Most-frequent-tag baseline about 90% (many unambiguous words: *the*, punctuation). Trigram HMM about 95% (55% on unknown words). TnT (Brants) slightly higher. MaxEnt tagger (Toutanova & Manning) about 97%; BiLSTM-CRF/BERT about 97.5%. Manning (2011): of the remaining errors, about 60% are underspecified, unclear, or gold errors. Common confusions: JJ/NN (*official* knowledge), RP/IN (made *up*), VBD/VBN (recently *sold* shares).

19.4 Exercises

Canvas Exercise 7.0

Tags: Noun, Verb, Adjective, Adverb, Determiner, Conjunction, Pronoun, Preposition, Particle, Cardinal. Sentence "I make small gains on my investments". Possible tags for *gains* (any context):

Answer: Noun, Verb. Most likely here:

Answer: Noun. Possible for *make*:

Answer: Noun, Verb ("the make of the car"). Most likely full tagging:

Answer: N V ADJ N Prep N N. (The answer key tags *I* and *my* as N in these options.)

Canvas Exercise 7.1

Classify *interest* in "Fed raises interest rates 0.5 percent" as noun/verb. Does a sentence-level unigram BoW work?

Answer: No: it doesn't capture the position of *interest* or its neighbors. Effective features:

Answer: Indicator(CurrWord="interest"). Not Contains("interest"), ThirdWord, or CurrWord=Noun. Other useful information:

Answer: prefixes/suffixes, context words, seen before or not, word length, contains digits (all of them).

Traps

- Count readings by multiplying the number of possible tags per word.
- "Possible tag" means in *some* context; "most likely" means in *this* context.
- Noun–noun compounds are normal (interest/NN rates/NNS). Don't force adjectives.
- Tagging features must be position-specific.

19.5 Practice

Practice

1. "Time flies like an arrow": give two readings with different tags for *flies*/*like*. Tag options: time {N, V}, flies {N, V}, like {V, P}, an {D}, arrow {N}. How many tag sequences?
2. Why does a per-word classifier with only CurrWord features get about 90% but fail on "can" in "they can fish"?

Answers

1. time/N flies/V like/P an/D arrow/N (normal); time/N flies/N like/V an arrow ("time-flies enjoy an arrow"). $2 \cdot 2 \cdot 2 \cdot 1 \cdot 1 = 8$ sequences.
2. CurrWord only gives each word type one most-likely tag. *can* as modal/verb vs. noun needs context (previous word "they").

TOPIC 20

Hidden Markov Models: Definition, Estimation, Viterbi

Sources: Lectures 7.2.0 Hidden Markov Models, 7.2.1 HMMs: Parameter Estimation, 7.3.0 HMMs: Viterbi Algorithm, 7.3.1 HMMs for POS Tagging; slides seg-24–seg-26; Exercises 7.2, 7.3; Fall 2022 Q1(2), Q1(8–10); Fall 2023 Q1(19–21). **De-emphasized this year (especially Viterbi), but in scope.**

In plain words

An HMM tells a *generative story*: pick a first tag, emit a word from it, move to a next tag (depending only on the current tag), emit a word, ..., then STOP. **Parameters** = three tables of probabilities (initial, transition, emission), estimated by **counting and normalizing** tagged data. **Tagging** = find the most probable tag sequence for a sentence. **Viterbi** dynamic programming does it in $O(nk^2)$ instead of checking all k^n sequences.

20.1 Definition

HMM joint probability

Tags $y_i \in \mathcal{T}$, words $x_i \in V$.

$$P(\mathbf{y}, \mathbf{x}) = P(y_1) \prod_{i=2}^{n} P(y_i \mid y_{i-1}) \cdot P(\text{STOP} \mid y_n) \cdot \prod_{i=1}^{n} P(x_i \mid y_i)$$

name	distribution	shape	meaning
initial $\mathbf{s}$	$P(y_1)$	$\lvert\mathcal{T}\rvert$	which tag starts a sentence
transitions T	$P(y_i \mid y_{i-1})$	$\lvert\mathcal{T}\rvert \times (\lvert\mathcal{T}\rvert + 1)$	next tag (incl. STOP) given current tag
emissions E	$P(x_i \mid y_i)$	$\lvert\mathcal{T}\rvert \times \lvert V\rvert$	word **given** tag

Each row is a probability distribution (entries in $[0,1]$ summing to 1). Transitions don't depend on position i. **Assumptions:** (1) Markov: $y_i \perp y_{1:i-2} \mid y_{i-1}$. (2) Each word depends only on its own tag (so "San Francisco" and "San York" are equally plausible as NNP NNP). **Generative:** models $P(\mathbf{y}, \mathbf{x})$, unlike discriminative LR's $P(y|\mathbf{x})$.

Midterm Fall 2022 Q1(2)

HMM tag transition probabilities are:

Official answer: D. the probability of a POS tag given the preceding tag, not conditioned on the words.

20.2 Parameter estimation: count and normalize

Maximize the generative log-likelihood $\sum_i \log P(\mathbf{y}^{(i)}, \mathbf{x}^{(i)})$. The MLE is relative frequency, like a biased coin: H H H T gives $\hat{p} = 3/4$ (maximizes $3 \log p + \log(1-p)$). No gradient descent needed.

Count and normalize

1. **Initial:** count first tags → divide by the number of sentences.
2. **Transitions:** count tag bigrams including y_n →STOP → divide each row by that tag's outgoing total.
3. **Emissions:** count (tag, word) pairs → divide by the tag's total count. This is $P(\text{word}|\text{tag})$, *not* $P(\text{tag}|\text{word})$.
4. Optional **smoothing**: add pseudo-counts (e.g. +1) so unseen events aren't impossible.

Worked example: lecture: "they can" and "they fish", both N V

Initial: N 2, V 0, so $P(N) = 1$, $P(V) = 0$. Transitions: N→V 2, V→STOP 2, so $P(V|N) = 1$, $P(\text{STOP}|V) = 1$. Emissions: $P(\text{they}|N) = 1$; $P(\text{can}|V) = P(\text{fish}|V) = \frac{1}{2}$ (probability of the *word* given V). **Add-one smoothing on transitions:** N row counts (N 1, V 3, STOP 1), V row (N 1, V 1, STOP 3), giving N: $(\frac{1}{5}, \frac{3}{5}, \frac{1}{5})$ and V: $(\frac{1}{5}, \frac{1}{5}, \frac{3}{5})$. $P(\text{they/N can/V fish/V}) = 1 \cdot 1 \cdot \frac{3}{5} \cdot \frac{1}{2} \cdot \frac{1}{5} \cdot \frac{1}{2} \cdot \frac{3}{5} = \frac{9}{500}$ (Exercise 7.2 Q1). Without smoothing the V→V transition would be 0. Exercise 7.2 Q2: no other tag sequence for "they can fish" has nonzero probability (they must be N, can/fish must be V).

20.3 Inference: Viterbi

$\arg\max_{\mathbf{y}} P(\mathbf{y}|\mathbf{x}) = \arg\max_{\mathbf{y}} \frac{P(\mathbf{y},\mathbf{x})}{P(\mathbf{x})} = \arg\max_{\mathbf{y}} \log P(\mathbf{y}, \mathbf{x})$, since $P(\mathbf{x})$ is constant in $\mathbf{y}$.

Viterbi recurrence (log space)

$v_i(y)$ = score of the best path ending in tag y at position i (chart $n \times |\mathcal{T}|$).

$$
\begin{aligned}
v_1(y) &= \log P(y) + \log P(x_1 \mid y) \\
v_i(y) &= \log P(x_i \mid y) + \max_{y'} \left[v_{i-1}(y') + \log P(y \mid y') \right] \quad \text{(store backpointer } y'\text{)} \\
\text{best} &= \max_{y} \left[v_n(y) + \log P(\text{STOP} \mid y) \right], \text{ then follow backpointers.}
\end{aligned}
$$

Works with any real-valued scores (CRFs too). Probability space: replace $+$ with $\times$. **Runtime** $O(nk^2)$ for n words and k tags (each of nk cells maxes over k predecessors). It implicitly searches $O(k^n)$ paths. If each tag can be followed by at most l tags, there are $O(kl^{n-1})$ paths. **Why DP works:** the Markov property means a cell needs only the previous column. Keep *all* tags alive at each step, because later words can change which earlier tag is best.

Worked example: Exercise 7.3 (log probabilities)

$\log P(y_1)$: N -1, V -1. Transitions (to N, V, STOP): from N $(-2, -1, -1)$; from V $(-1, -1, -2)$. Emissions (they, can, fish): N $(-1, -3, -1)$; V $(-3, -1, -1)$.

	they	can	fish
N	$-1-1=\mathbf{-2}$	$-3+\max(-2-2,-4-1)=\mathbf{-7}$ (from N)	$-1+\max(-7-2,-4-1)=\mathbf{-6}$ (from V)
V	$-1-3=\mathbf{-4}$	$-1+\max(-2-1,-4-1)=\mathbf{-4}$ (from N)	$-1+\max(-7-1,-4-1)=\mathbf{-6}$ (from V)

STOP: N $-6-1=-7$; V $-6-2=-8$. Best score

Answer: -7; backpointers N←V←N give

Answer: N V N.

Midterm Fall 2022 Q1(8)–(10)

Initial J 2/3, N 1/3. Transitions (to J, N, STOP): from J $(1/2, 1/4, 1/4)$; from N $(0, 1/2, 1/2)$. Emissions: J: big 1/2, red 1/4, convertible 1/4, car 0; N: big 0, red 0, convertible 1/3, car 2/3.
(8) $P(\text{J/big N/car}) = \frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{4}\cdot\frac{2}{3}\cdot\frac{1}{2} =$

Official answer: 1/36.
(9) Tag sequences with nonzero probability for "big convertible car": big must be J and car must be N, so J J N and J N N:

Official answer: 2.
(10) "big car convertible": J N ?; N→J is 0, so only J N N:

Official answer: 1.
Viterbi on (9) (probability space): v_1: J 1/3, N 0. v_2: J $= \frac{1}{3}\cdot\frac{1}{2}\cdot\frac{1}{4} = \frac{1}{24}$; N $= \frac{1}{3}\cdot\frac{1}{4}\cdot\frac{1}{3} = \frac{1}{36}$. v_3: N $= \frac{2}{3}\max(\frac{1}{24}\cdot\frac{1}{4}, \frac{1}{36}\cdot\frac{1}{2}) = \frac{2}{3}\cdot\frac{1}{72} = \frac{1}{108}$ (from N). With STOP, $\frac{1}{216}$. Best: **J N N** ($1/216 > 1/288$ for J J N).

Midterm Fall 2023 Q1(19)–(21)

Viterbi runtime:

Official answer: $O(nk^2)$. Paths searched:

Official answer: $O(k^n)$. If each tag can only be followed by l tags:

Official answer: $O(kl^{n-1})$.

20.4 HMMs for POS tagging in practice

A **trigram HMM** uses tag-pair states (y_{i-1}, y_i), so transitions condition on two previous tags and the Viterbi algorithm is unchanged. Trigrams are the sweet spot. About 95% on the Penn Treebank vs. 90% for the most-frequent-tag baseline.

Traps

- Emission direction: $P(\text{word} \mid \text{tag})$, normalized over words for a fixed tag.
- Don't forget the STOP transition at the end, and the initial distribution at the start.
- A single zero factor (unsmoothed) makes the whole sequence probability 0.
- Viterbi takes a *max* (best path); summing instead gives the forward algorithm ($P(\mathbf{x})$, not on this exam).
- Keep the best score *per tag* at each position; don't greedily commit.

20.5 Practice

Practice

1. Estimate the HMM (initial, transitions incl. STOP, emissions) from: they/N can/V fish/N; fish/N can/V fish/V; they/N fish/V.
2. Using your estimates (no smoothing), compute $P(\text{they/N can/V fish/V})$ and $P(\text{they/N can/V fish/N})$. Which does Viterbi choose?
3. With 45 tags and a 25-word sentence, how many cells does the Viterbi chart have, and roughly how many operations?

Answers

1. Initial: N 1. Transitions: from N: V 3, STOP 1, so $(N\,0, V\,\frac{3}{4}, \text{STOP}\,\frac{1}{4})$. From V: N 1, V 1, STOP 2, so $(\frac{1}{4}, \frac{1}{4}, \frac{1}{2})$. Emissions: N: they $\frac{1}{2}$, fish $\frac{1}{2}$; V: can $\frac{1}{2}$, fish $\frac{1}{2}$.
2. N V V: $1 \cdot \frac{1}{2} \cdot \frac{3}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{256}$. N V N: $1 \cdot \frac{1}{2} \cdot \frac{3}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{1}{4} = \frac{3}{512}$. Viterbi picks **N V V**.
3. $25 \times 45 = 1125$ cells; about $25 \times 45^2 \approx 50{,}625$ operations.

TOPIC 21

Constituency Syntax: Trees and Ambiguity

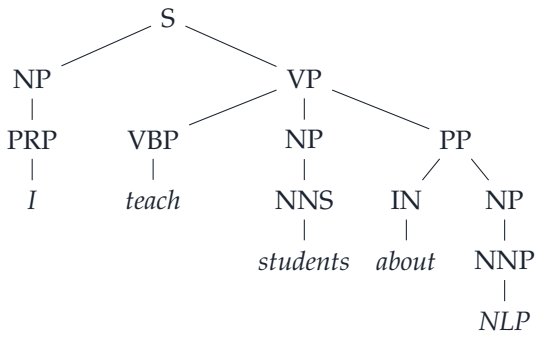
Sources: Lecture 7.4.0 Constituency Parsing; slides seg-28, seg-34; Fall 2023 Q4(b,d); Fall 2022 Q4(a). Dependency parsing is **not** on this exam.

In plain words

Constituency trees group words into nested phrases: noun phrases (NP), verb phrases (VP), prepositional phrases (PP), sentences (S). The leaves are words, the layer above them is POS tags (pre-terminals), and phrases sit above that. The same word string can have different trees with different meanings. Most famously, a PP can attach to a verb or to a noun. The grammar allows both; choosing needs world knowledge, which is why parsing is hard.

21.1 What trees look like

- Root S; phrases NP, VP, PP, ADJP, ADVP, SBAR (sentential complement: "she told me [that I would never amount to anything]"); bottom layer = POS tags.
- Nodes can have any arity: unary (NP → PRP), binary, ternary (VP → VBD NP PP). English is mostly **right-branching**.
- Bracket notation: `(S (NP (PRP I)) (VP (VBP teach) (NP (NNS students)) (PP (IN about) (NP (NNP NLP)))))`.
- Penn Treebank trees also have traces/co-indexation ("a refund that the court estimated *T*-1"), which strictly makes them graphs. We ignore this.
- Constituency suits languages with fairly fixed word order. Free-word-order languages are better served by dependency grammars.



21.2 Ambiguities (must recognize)

type	example and readings
PP attachment	*The children ate the cake with a spoon*: [VP ate the cake][PP with a spoon] (instrument) vs. [NP the cake [PP with a spoon]] (cake that has a spoon, like "cake with icing"). *Eat spaghetti with chopsticks* (VP attachment, instrument) vs. *eat spaghetti with meatballs* (NP attachment). Same tag sequence and same rule types; only the words differ.
PP attachment in headlines	*Ban on nude dancing on Governor's desk*: is the *ban* on the desk, or the *dancing*? [NP ban [PP on nude dancing]] [PP on desk] attaches to ban vs. to dancing.
Modifier scope	*plastic cup holder*: [[plastic cup] holder] (holder for plastic cups) vs. [plastic [cup holder]] (cup holder made of plastic).
Complement structure	*The students complained to the professor that they didn't understand*: relative clause on professor vs. content of the complaint.
Coordination scope	*The man picked up his hammer and saw*: [hammer and saw] (nouns) vs. [picked up his hammer] and [saw] (verb).
POS-driven	*Fed raises interest rates* (Topic 19): different tags give different trees.

PP attachment as trees

VP attachment: (VP (VP (V ate) (NP spaghetti)) (PP (P with) (NP chopsticks))) or (VP V NP PP).
NP attachment: (VP (V ate) (NP (NP spaghetti) (PP (P with) (NP meatballs)))).
A plain PCFG scores these with *rule* probabilities only, so it prefers the same attachment for chopsticks and meatballs (Topic 22). Fixing that needs lexical information.

21.3 Constituency tests (how linguists justify phrases)

- **Substitution by pro-form:** "*They* ate the cake with a spoon" (the children = NP); "the children *did so* with a spoon" (ate the cake = VP).
- **Clefting:** "It was *with a spoon* that the children ate the cake."
- **Answer to a question:** What did they eat? *The cake.* How? *With a spoon.*
- Limits: "She went to and bought food at the store" (*the store* serves both prepositions), so constituency isn't perfectly clean.

Midterm Fall 2022 Q4(a)

Grammar: S→PRP VP; VP→V NP; NP→J N; lexicon I, like/eat, green/red/verde/roja, salsa. Parse "I like green salsa".

Official answer: (S (PRP I) (VP (V like) (NP (J green) (N salsa)))).

Traps

- Write brackets carefully: every node is `(LABEL children...)`; pre-terminals wrap exactly one word.
- Identify which phrase a PP attaches to: a VP-sister PP modifies the verb; an NP-internal PP modifies the noun.
- You don't need encyclopedic grammar. Use the grammar or tags the question gives.

21.4 Practice

Practice

1. Give both bracketed parses of “I saw the man with the telescope” (use S, NP, VP, PP, PRP, V, DT, N, P) and say what each means.
2. For “Ban on nude dancing on Governor’s desk”, write the NP structure for the reading where the dancing is on the desk.

Answers

1. VP attachment (I used the telescope): (S (NP (PRP I)) (VP (VP (V saw) (NP (DT the) (N man))) (PP (P with) (NP (DT the) (N telescope))))). NP attachment (the man has the telescope): (S (NP (PRP I)) (VP (V saw) (NP (NP (DT the) (N man)) (PP (P with) (NP (DT the) (N telescope)))))).
2. (NP (NP Ban) (PP on (NP (NP nude dancing) (PP on (NP Governor’s desk))))). The second PP sits inside the NP headed by *dancing*.

TOPIC 22

PCFGs, Binarization, Markovization, CKY

Sources: Lectures 7.4.1 Probabilistic Context-Free Grammars, 7.4.2 CKY Algorithm, 7.4.3 Refining Grammars; slides seg-34–seg-39; Exercises 7.4, 7.5; Fall 2022 Q4; Fall 2023 Q4.

In plain words

A PCFG is a context-free grammar whose rules have probabilities (for each left-hand symbol, its rules sum to 1). A tree's probability is the product of its rules' probabilities. Get the probabilities from a treebank by **counting and normalizing** rules, after **binarizing** trees so every rule has at most two children. Find the best tree with **CKY**, a dynamic program over spans, $O(n^3|G|)$. Plain PCFGs are too "context-free", so **Markovization** (annotating parents, limiting sibling memory) refines the symbols and boosts accuracy a lot.

22.1 CFG and PCFG

Definitions

CFG $= (N, T, S, R)$: non-terminals N (S, NP, VP, PP, … plus POS tags, which are **pre-terminals**), terminals T (words), start symbol S, rules R. Rules: binary (NP $\to$ DT NN), unary non-terminal (NP $\to$ PRP), and lexicon (NN $\to$ cake).
PCFG: each rule has $P(\text{rule} \mid \text{parent})$, and **probabilities sum to 1 per parent**: $\sum_\beta P(A \to \beta) = 1$.
$P(\text{tree}) = \prod_{\text{rules } r \in \text{tree}} P(r \mid \text{parent}(r))$.

Midterm Fall 2022 Q4(b)–(e)

(b) P((S (PRP I) (VP (V like) (NP (J green) (N salsa))))) $= 1 \cdot 1 \cdot 1 \cdot 1 \cdot 0.5 \cdot 0.25 \cdot 1 =$

Official answer: 1/8 (S, PRP→I, VP, V→like 0.5, NP, J→green 0.25, N→salsa).
(c) Add one non-terminal rule to parse "I like salsa verde":

Official answer: NP $\to$ N J.
(d) An ungrammatical sentence it now generates:

Official answer: "I like salsa red".
(e) Split Spanish/English:

Official answer: NP-EN $\to$ J-EN N-EN; NP-ES $\to$ N-ES J-ES; J-EN $\to$ green | red; J-ES $\to$ verde | roja; N-EN $\to$ salsa; N-ES $\to$ salsa (splitting N is optional). You'd also need VP $\to$ V NP-EN | V NP-ES.

Canvas Exercise 7.4

PCFG: NP→NP CC NP [0.3]; NP→NP PP [0.3]; NP→NNS [0.4]; PP→P NP [1]; NNS→cats; CC→and; P→in; root NP. "cats and cats":

Answer: 1 parse. "cats and cats in cats":

Answer: 2 parses: [[cats and cats] in cats] vs. [cats and [cats in cats]] (coordination vs. PP attachment scope).

22.2 From treebank to grammar

Pipeline

1. **(Optional) Markovize / annotate** the trees (below).
2. **Binarize** so every rule is binary or unary.
3. **Estimate:** count each rule and normalize by its parent's total. Example: NP→X seen 3 times and NP→Y seen 4 times gives 3/7 and 4/7. This is the MLE (same multinomial logic as HMMs).
4. **Parse** new sentences with CKY.

Midterm Fall 2023 Q4(a) — extract a grammar (no lexicon, no binarization)

Trees: (S (NP (PRP I)) (VP (VBP teach) (NP (NNS students)) (PP (IN about) (NP (NNP NLP))))) and (S (NP (PRP he)) (VP (VBP gives) (NP (NNS lessons)))).

Official answer: S→NP VP; NP→PRP; NP→NNS; NP→NNP; VP→VBP NP PP; VP→VBP NP; PP→IN NP. With counts: S→NP VP 2/2 = 1; NP→PRP 2/5, NP→NNS 2/5, NP→NNP 1/5; VP→VBP NP PP 1/2, VP→VBP NP 1/2; PP→IN NP 1.
(b) Parse "him teach I": (S (NP (PRP him)) (VP (VBP teach) (NP (PRP I)))). **Erratum:** "him" was accidentally missing from the lexicon, so "not parseable" was also accepted. The point: in this grammar the accusative *him* can be a subject, since NPs are interchangeable.

22.3 Binarization

A 4-ary rule like VP → VBD NP PP PP ("sold the book to her for $3") makes estimation sparse and parsing slower (CKY's cost grows with rule arity).

Lossless vs. lossy

Lossless: introduce intermediate symbols that remember what remains: VP → VBD [VP→VBD·NP PP PP], … The first rule carries probability p; the intermediate rules have probability 1, since the symbol already encodes the yield. Equivalent (chain rule) form: generate children one at a time, each conditioned on everything generated so far.
Lossy (e.g. reuse plain VP for intermediates): intermediates compete with real VPs for probability mass, so "VP" learns to spit out PPs. This changes the model.

22.4 Markovization (refining grammars)

PCFGs are **too context-free**. NP → PRP is 21% for subject NPs (under S) but only 4% for object NPs (under VP); overall NP→NP PP is 11%, DT NN 9%, PRP 6%.

Vertical and horizontal Markovization

Vertical (parent annotation, order v): relabel each node with its parent: NP under S becomes **NP^S**, NP under VP becomes **NP^VP**. Applied to the treebank *before* extracting the grammar. Captures context-dependent rule distributions; grammar gets bigger.
Horizontal (order h, during binarization): intermediate symbols remember only the last h generated siblings. $h = 0$ is like lossy (most compact, weakest); $h = 1$ remembers the previous sibling; $h = \infty$ is fully lossless. The head is usually always remembered.

Klein & Manning (2003): going from $v = 1, h = 0$ (≈70 F1) to good v, h settings gives ≈80 F1. Lexicalized parsers (Collins, Charniak) ≈90 F1 annotate each node with its **head word** (VP → verb, NP → noun, S → verb). That fixes attachments that depend on words (dogs in houses and cats), but makes the grammar huge and the parsers very complex.

Midterm Fall 2023 Q4(c)–(e) — vertical Markovization of NPs only

(c) New grammar:

Official answer: S→NP^S VP; NP^S→PRP; NP^VP→NNS; NP^PP→NNP; VP→VBP NP^VP PP; VP→VBP NP^VP; PP→IN NP^PP.

(d) Parse "him teach I"?

Official answer: Not parseable: PRP can only appear under NP^S, and the object position needs NP^VP → NNS.

(e) The language of the Markovized grammar vs. the original:

Official answer: a subset, since NPs can no longer be swapped between subject and object positions.

22.5 CKY

CKY recurrence (log probabilities, binary grammar)

Chart $t(i, j, X)$ = score of the best derivation of symbol X over span $[i, j)$ (fenceposts $0..n$).

$$\textbf{Base: } t(i, i+1, X) = \log P(X \to w_{i+1})$$
$$\textbf{Recurrence: } t(i, j, X) = \max_{k: i<k<j} \max_{X \to X_1 X_2} \left[\log P(X \to X_1 X_2) + t(i, k, X_1) + t(k, j, X_2)\right]$$

Answer: $t(0, n, S)$; follow backpointers (split k and rule) to recover the tree. $\arg\max_T P(T|\mathbf{x}) = \arg\max_T P(T, \mathbf{x})$. **Runtime** $O(n^3|G|)$: n^2 cells × $O(n)$ split points × grammar constant. Unary rules need an extra closure step (not covered). Fill the chart by increasing span length.

Worked example: lecture CKY: "the child raises it" ($\log_2$: $\log 1 = 0, \log 0.5 = -1$)

Grammar: DT→the, NN→child, NNS→raises, VBZ→raises, PRP→it (all 1.0); S→NP VP 1.0; NP→DT NN 0.5; NP→NN NNS 0.5; VP→VBZ PRP 1.0.

span	entries	how
$[0,1]$ the	DT: 0	lexicon
$[1,2]$ child	NN: 0	
$[2,3]$ raises	NNS: 0, VBZ: 0	two tags
$[3,4]$ it	PRP: 0	
$[0,2]$	NP: -1	NP→DT NN: $-1+0+0$
$[1,3]$	NP: -1	NP→NN NNS (a dead end)
$[2,4]$	VP: 0	VP→VBZ PRP
$[0,3]$, $[1,4]$	—	no rule applies
$[0,4]$	S: -1	split $k=2$: S→NP[0,2] VP[2,4] $= 0-1+0$

Tree: (S (NP (DT the) (NN child)) (VP (VBZ raises) (PRP it))), probability $2^{-1} = 0.5$.

Canvas Exercise 7.5

Same grammar. Best parse of "child raises raises it":

Answer: (S (NP (NN child) (NNS raises)) (VP (VBZ raises) (PRP it))), score -1. "the child raises raises" has no parse. Which single added rule fixes it?

Answer: VP→VBZ NNS, or PRP→raises. (Not NP→NNS VBZ; not S→NP NP.)

Worked example: why a plain PCFG can't resolve PP attachment

Grammar: S→PRP VP 1; VP→V NP 0.7; VP→VP PP 0.3; NP→NP PP 0.2; PP→P NP 1; NP→spaghetti/chopsticks/meatballs (0.32/0.24/0.24). "I ate spaghetti with chopsticks": VP-attach $= 0.3 \cdot 0.7 \cdot 0.32 \cdot 0.24 = 0.016128$; NP-attach $= 0.7 \cdot 0.2 \cdot 0.32 \cdot 0.24 = 0.010752$, so VP-attach wins. "…with meatballs": exactly the same numbers, so VP-attach wins again, which is wrong. Rule probabilities don't see the words, so lexicalization or a discriminative parser is needed.

Traps

- Rule probabilities normalize over rules with the *same parent*, not over all rules.
- Tree probability = product over *all* rules, including lexicon rules (e.g. V→like 0.5).
- Grammar-extraction questions: follow the requested order (S, NP, VP, PP), exclude the lexicon if told, and list each distinct rule once.
- Vertical Markovization makes a grammar that generates a *subset* of the original (more specific symbols).
- CKY needs a binarized grammar; spans use fenceposts; split point k is strictly between i and j.
- Log base: use the one stated (the lecture CKY uses $\log_2$).

22.6 Practice

Practice

1. Losslessly binarize VP → VBD NP PP PP with $h = \infty$, then with $h = 1$ (write the intermediate symbols).
2. Apply vertical Markovization ($v = 2$, all non-pre-terminal nodes) to (S (NP (DT the) (NN dog)) (VP (VBD saw) (NP (PRP me)))). List the extracted non-lexical rules.
3. How many parses does "cats and cats and cats" have under Exercise 7.4's grammar?

Answers

1. $h = \infty$: VP→VBD @VP[VBD]; @VP[VBD]→NP @VP[VBD,NP]; @VP[VBD,NP]→PP PP (each intermediate remembers all previous siblings). $h = 1$: VP→VBD @VP[VBD]; @VP[VBD]→NP @VP[NP]; @VP[NP]→PP PP. Only the last sibling is remembered, so @VP[NP] is shared with other VPs that just generated an NP.
2. S→NP^S VP^S; NP^S→DT NN; VP^S→VBD NP^VP; NP^VP→PRP.
3. 2: [[cats and cats] and cats] and [cats and [cats and cats]] (NP→NP CC NP applies at two levels).